

Flutter từ cơ bản đến nâng cao — Tập 3: Nâng cao

Kiến trúc feature-first, performance, platform channel, bảo mật, CI/CD, phát hành, monitoring

Flutter 3.47.5 · Dart 3.13.4 · provider 6.1.5+1 · go_router 18.0.2 · sqflite 2.4.4

Bản dựng: 2026-09-30 — Markdown là nguồn chính (books/flutter/vol3-nang-cao/)

Tham khảo docs.flutter.dev (CC BY 3.0) và dart.dev (CC BY 4.0) — xem Nguồn tham khảo cuối mỗi chương

Kiểm tra dấu tiếng Việt: ă ằ ẳ ẵ ơ ư ơ đ ư đ — Đường ướn, trắng khuyết, Ưu tiên

Mục lục

Chương 1 — Kiến trúc: feature-first, các tầng, và luật phụ thuộc

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 2 — Performance: đo trước, sửa sau

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 3 — Platform channels và plugin: gọi Swift/Kotlin từ Dart

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 4 — Bảo mật: lưu bí mật, PIN, khóa tự động, deep link, obfuscation

Mục tiêu

Giải thích đơn giản

Ví dụ

Đi sâu

Lỗi và bẫy thường gặp

Tóm tắt

Bài tập (có lời giải)

Nguồn tham khảo (Sources)

Chương 5 — CI/CD: tự động kiểm tra, build và giao app

Mục tiêu

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 6 — Phát hành App Store và Google Play](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 7 — Monitoring: bắt lỗi toàn app, logging, báo cáo crash](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[Chương 8 — App mẫu Tập 3: "Sổ Ghi Chú Bảo Mật"](#)

[Mục tiêu](#)

[Giải thích đơn giản](#)

[Ví dụ](#)

[Đi sâu](#)

[Lỗi và bẫy thường gặp](#)

[Tóm tắt](#)

[Bài tập \(có lời giải\)](#)

[Nguồn tham khảo \(Sources\)](#)

[GLOSSARY — Thuật ngữ Flutter \(Anh → Việt\)](#)

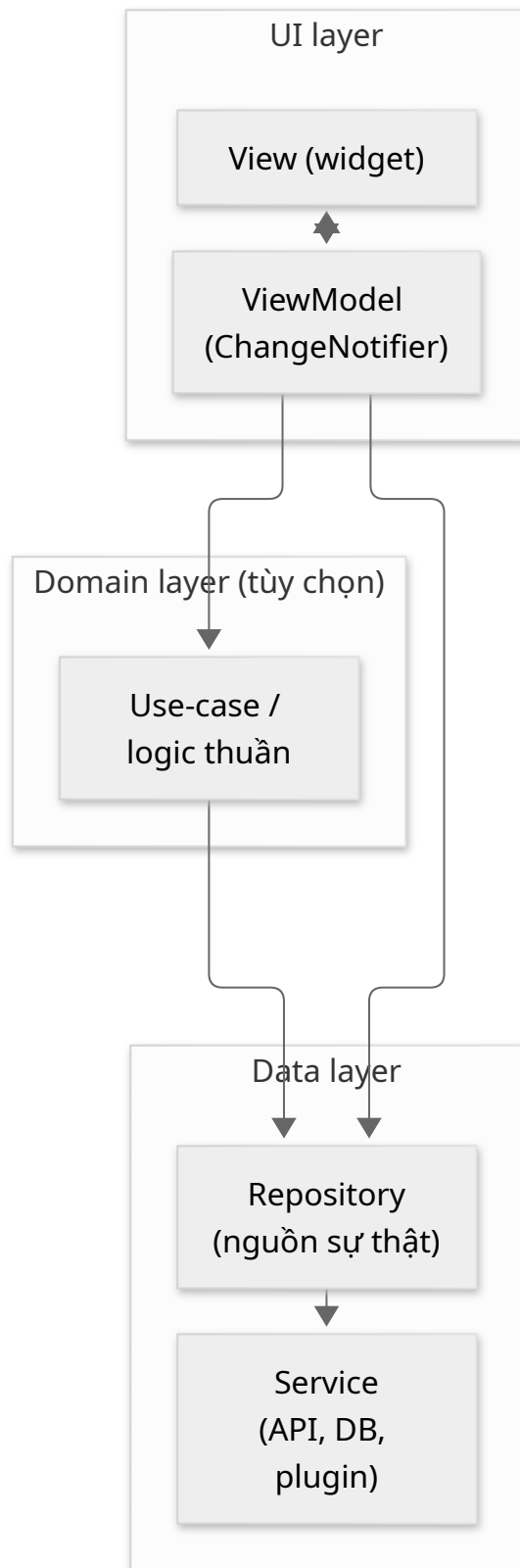
Chương 1 — Kiến trúc: feature-first, các tầng, và luật phụ thuộc

Mục tiêu

- Nắm kiến trúc chính thức của Flutter: **UI layer** (View + ViewModel), **data layer** (Repository + Service), **domain layer** (tùy chọn), luồng dữ liệu một chiều, DI.
- So sánh hai cách xếp thư mục: **theo loại (by type)** và **theo tính năng (feature-first)** — và cách **kết hợp** mà docs khuyên.
- Viết **test kiến trúc**: kiểm tra luật phụ thuộc bằng code.
- Biết khi nào cần domain layer / use-case.

Giải thích đơn giản

Docs "Guide to app architecture" chia app thành các tầng:



- **View**: chỉ hiển thị và chuyển sự kiện; logic duy nhất được phép là `if` đơn giản, animation, layout theo kích thước, điều hướng đơn giản.
- **ViewModel**: giữ state của màn hình, gọi repository, biến dữ liệu thành thứ View cần.
- **Repository**: nguồn sự thật cho một loại dữ liệu; cache, retry, kết hợp nhiều nguồn.
- **Service**: bọc một nguồn bên ngoài (HTTP, SQLite, Keychain, plugin); không có state.

- **Domain / use-case:** docs nói **chỉ cần** khi logic phức tạp hoặc lặp lại giữa nhiều ViewModel.

Với Angular dev: View $\approx$ component template, ViewModel $\approx$ component class + facade service, Repository $\approx$ data service, Service $\approx$ `HttpClient` wrapper.

Xếp thư mục: theo loại hay theo tính năng?

Cách	Ví dụ	Ưu	Nhược
Theo loại (by type)	<code>repositories/</code> , <code>viewmodels/</code> , <code>screens/</code>	Dễ tìm "mọi repository"	Một tính năng nằm rải rác nhiều thư mục
Feature-first	<code>features/auth/{data, domain, ui}</code> , <code>features/notes/...</code>	Một tính năng ở một chỗ; dễ xóa/giao cho người khác	Dữ liệu dùng chung giữa tính năng bị "kẹt" trong một feature
Kết hợp (docs khuyến)	<code>ui/<feature>/</code> , <code>data/repositories/</code> , <code>data/services/</code> , <code>domain/models/</code>	UI theo tính năng (mỗi màn một View + một ViewModel); data theo loại (dùng chung)	—

Docs (case study Compass app, phần "Package structure"): kiến trúc được khuyến **kết hợp cả hai** — data layer theo loại vì repository/service dùng chung nhiều tính năng; UI layer theo tính năng.

Trong bộ sách:

- App Tập 2 "Sổ Từ Vựng" dùng **đúng cấu trúc kết hợp của docs** → **app TOEIC (Task 9) theo cấu trúc này**.
- App Tập 3 "Sổ Ghi Chú Bảo Mật" dùng **feature-first thuần** (đề bài yêu cầu) để bạn thấy cách còn lại, kèm test luật phụ thuộc.

Ví dụ

Cấu trúc feature-first của app Tập 3

```
examples/lib/  
  main.dart  app.dart  router.dart  
  core/      dùng chung, KHÔNG phụ thuộc feature nào  
    result.dart  command.dart  (mẫu chính thức, BSD)  
    logging/     app_logger, error_handlers, redact  
    platform/battery_channel.dart  
    security/secure_store.dart  
    ui/theme.dart  
  features/  
    auth/  
      domain/pin_hasher.dart      logic thuần: bấm PIN, khóa tạm, PIN yếu, tự khóa  
      data/pin_repository.dart    lưu muối + hash trong SecureStore  
      ui/auth_controller.dart     ChangeNotifier: needsSetup / locked / unlocked  
      ui/lock_screen.dart  
    notes/  
      domain/note.dart  
      data/note_repository.dart    interface + SecureNoteRepository  
      ui/notes_viewmodel.dart     NotesViewModel, NoteEditorViewModel (Command)  
      ui/notes_list_screen.dart   ui/note_editor_screen.dart  
  settings/ui/settings_screen.dart
```

Luật phụ thuộc — kiểm tra bằng test

```
examples/lib/chapters/ch01/dependency_rules.dart:
```

```

List<String> checkDependencyRules(Map<String, String> sources) {
  final violations = <String>[];
  final importRe = RegExp(r''^import\s+["']([^"']+)["']'', multiline: true);
  for (final MapEntry(key: path, value: code) in sources.entries) {
    final parts = path.split('/');
    if (parts.length < 3 || parts[0] != 'features') continue;
    final feature = parts[1];
    final layer = parts[2];
    for (final m in importRe.allMatches(code)) {
      final target = m.group(1)!;
      final isUiImport =
        target == 'package:flutter/material.dart' ||
        target == 'package:flutter/widgets.dart' ||
        target.contains('/ui/') ||
        target.startsWith('../ui/');
      if ((layer == 'data' || layer == 'domain') && isUiImport) {
        violations.add('$path: tầng $layer không được import UI ($target)');
      }
      final other =
        RegExp(r'features/(w+)/data/').firstMatch(target) ??
        RegExp(r'\.\.\./(\w+)/data/').firstMatch(target);
      if (other != null && other.group(1) != feature) {
        violations.add('$path: feature "$feature" import data của feature
"${other.group(1)}" ($target)');
      }
    }
  }
  return violations;
}

```

Test đọc **toàn bộ** lib/ thật và yêu cầu không có vi phạm:

```

test('mã thật của app không vi phạm', () {
  final lib = Directory('lib');
  final sources = {
    for (final f in lib.listSync(recursive: true).whereType<File>().where((f) =>
      f.path.endsWith('.dart')))
      f.path.substring(lib.path.length + 1).replaceAll(r'\', '/'):
      f.readAsStringSync(),
  };
  expect(sources.keys, contains('features/auth/data/pin_repository.dart'));
  expect(checkDependencyRules(sources), isEmpty);
});

```

Kết quả thật (2026-09-30, flutter test --reporter expanded test/chapters_test.dart, phần Ch.1):

```

00:00 +0: Ch.1 – luật phụ thuộc (feature-first) phát hiện vi phạm trong mã giả
00:00 +1: Ch.1 – luật phụ thuộc (feature-first) Bài 1: core không được import features
00:00 +2: Ch.1 – luật phụ thuộc (feature-first) mã thật của app không vi phạm

```


Khi đang viết app, sách định đặt hàm `isWeakPin` trong `chapters/` và gọi từ `features/auth/ui` — điều đó làm feature phụ thuộc vào code ví dụ. Sách chuyển hàm vào `features/auth/domain/pin_hasher.dart`; file lời giải trong `chapters/` chỉ `export` lại. Tương tự, `redact` chuyển vào `core/logging/` vì `logger` (ở `core`) cần nó.

DI ở gốc app

```
MultiProvider(  
  providers: [  
    Provider<NoteRepository>.value(value: dependencies.notes),  
    Provider<BatteryChannel>.value(value: dependencies.battery),  
    ChangeNotifierProvider<AppLogger>.value(value: dependencies.logger),  
    ChangeNotifierProvider<AuthController>(  
      create: (_) => AuthController(pins: dependencies.pins, logger:  
dependencies.logger)..init(),  
    ),  
  ],  
  child: const _AppView(),  
);
```

`AuthController` là **state toàn app** (docs gọi là "app-wide session state" — quản lý ở repository/controller dùng chung); router đọc nó để chặn màn hình khi chưa mở khóa (Chương 4).

Đi sâu

Command và Result

Docs khuyên dùng **Command** cho sự kiện người dùng (tránh bấm nhiều lần, có trạng thái running/error) và **Result** cho kết quả có thể lỗi. App Tập 2–3 dùng cả hai (`core/command.dart`, `core/result.dart`). View: `onPressed: vm.save.running ? null : () => _save(vm)`, và `switch (vm.save.result) { Ok() => ..., Error(:final error) => ... }`.

Khi nào thêm domain layer?

Khi cùng một logic xuất hiện ở nhiều ViewModel, hoặc ViewModel bị "phình". App Tập 3 có `auth/domain/pin_hasher.dart` vì logic bấm PIN, khóa tạm, tự khóa là **thuần** (không I/O) và cần test kỹ. Docs cảnh báo: với đa số app, use-case cho **mọi** thao tác chỉ thêm code thừa.

Model API vs model domain

Docs gợi ý tách model API (đúng JSON server) và model domain (đúng thứ UI cần) cho **app lớn**. App của sách nhỏ nên dùng một model.

Nhiều môi trường (flavor)

Case study có `main_development.dart`, `main_staging.dart`: mỗi file tạo bộ phụ thuộc khác nhau (repository giả / thật). `AppDependencies` của sách cho phép đúng điều đó (Chương 5, 6).

Lỗi và bẫy thường gặp

- **Logic trong widget** (gọi repository trong `onPressed`, tính toán trong `build`) → khó test. Đưa vào `ViewModel`.
- **Feature import data của feature khác** → vòng phụ thuộc; đưa phần dùng chung lên `core/` (hoặc theo cấu trúc kết hợp: `data/`).
- **ViewModel import `package:flutter/material.dart` để hiện `SnackBar`** → trộn tầng; `ViewModel` đặt `message`, `View` hiển thị.
- **Tạo use-case cho mọi thứ** → code thừa.
- **Không có test kiến trúc** → luật bị phá dần theo thời gian.

Tóm tắt

- Tầng: `View` ↔ `ViewModel` → (Use-case) → `Repository` → `Service`. Dữ liệu một chiều, DI bằng provider.
- Docs khuyên kết hợp: UI theo tính năng, data theo loại (app Tập 2, và **app TOEIC**). Feature-first thuần: app Tập 3.
- Kiểm tra luật phụ thuộc bằng test đọc mã nguồn.

Bài tập (có lời giải)

Bài 1. Thêm vào `checkDependencyRules` luật thứ 3: file trong `core/` **không** được import `features/`. Viết test.

▼ Lời giải

Trong `dependency_rules.dart`, xử lý file `core/` trước khi kiểm tra `features/`:

```
final parts = path.split('/');
if (parts.first == 'core') {
  for (final m in importRe.allMatches(code)) {
    if (m.group(1)!.contains('features/')) violations.add('$path: core không được
import features (${m.group(1)})');
  }
  continue;
}
```

Test "Bài 1: core không được import features": `{'core/x.dart': "import ' ../features/auth/data/pin_repository.dart';"} → 1 vi phạm. Test "mã thật của app không vi phạm" vẫn pass với luật mới — core/ chỉ import core/ và package.`

Bài 2. App TOEIC có các màn: Từ vựng, Thẻ ôn, Đọc hiểu, Hội thoại, Tiến độ, Cài đặt; dữ liệu: từ vựng, lịch sử ôn (SM-2), bài đọc, cài đặt. Hãy vẽ cây thư mục theo cấu trúc **kết hợp** của docs.

▼ Lời giải

```
lib/  
  main.dart  app.dart  
  config/dependencies.dart  config/router.dart  
  domain/models/           word.dart, review_card.dart, passage.dart, dialog.dart  
  data/services/           database_service.dart, tts_service.dart, reminder_service.dart,  
  key_value_store.dart  
  data/repositories/       word_repository.dart (+ sqlite...), review_repository.dart  
  (SM-2), passage_repository.dart,  
                           settings_repository.dart  
  ui/core/                 theme.dart, flip_card.dart, error_view.dart, word_popup.dart  
  (chạm từ → popup)  
  ui/words/  ui/review/  ui/reading/  ui/dialogs/  ui/progress/  ui/settings/  mỗi thư  
mục: *_screen.dart + *_viewmodel.dart  
  utils/                   result.dart, command.dart, vietnamese.dart  
  test/  data/  ui/  domain/  fakes/
```

Repository dùng chung (ví dụ `WordRepository` dùng ở Từ vựng, Đọc hiểu, Thẻ ôn) nằm ở `data/`, không nằm trong một feature — đúng lý do docs chọn cấu trúc kết hợp.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Guide to app architecture — <https://docs.flutter.dev/app-architecture/guide> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/guide.md>
- Common architecture concepts — <https://docs.flutter.dev/app-architecture/concepts> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/concepts.md>
- Case study (package structure) — <https://docs.flutter.dev/app-architecture/case-study> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/case-study/index.md>
- Architecture recommendations — <https://docs.flutter.dev/app-architecture/recommendations> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/recommendations.md>

- Design patterns (Command, Result) — <https://docs.flutter.dev/app-architecture/design-patterns> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/design-patterns.md>
- Compass app (mẫu chính thức): <https://github.com/flutter/samples>

Chương 2 — Performance: đo trước, sửa sau

Mục tiêu

- Hiểu ngân sách thời gian mỗi frame (16 ms ở 60 Hz, 8 ms ở 120 Hz) và vì sao phải đo ở **profile mode**.
- Giảm chi phí `build: const`, tách widget nhỏ, `select/Consumer`, tham số `child` của `AnimatedBuilder`.
- Danh sách lớn: `lazy (ListView.builder)`, `itemExtent`, tránh `shrinkWrap` và `intrinsic`.
- Việc nặng: đưa sang isolate (`compute`).
- Dùng DevTools (Performance, CPU profiler, Memory) và đo kích thước app.
- Chứng minh bằng test "đếm số lần build".

Giải thích đơn giản

Docs "Performance best practices": Flutter có luồng build (UI) và luồng vẽ (raster) riêng; ở màn hình 60 Hz bạn có khoảng **16 ms** cho mỗi frame; với thiết bị 120 Hz cần dưới **8 ms**. Quá thời gian → giật (jank). Docs nhấn mạnh: nếu frame đã dưới 16 ms ở **profile mode** thì thường không cần lo; nhưng làm nhanh hơn vẫn giúp **tiết kiệm pin**.

Quy trình đúng: **đo** → **tìm chỗ chậm** → **sửa** → **đo lại**. Đừng tối ưu theo cảm giác.

Angular	Flutter
<code>ChangeDetectionStrategy.OnPush</code>	<code>Widget const</code> , tách widget nhỏ, <code>select</code>
<code>trackBy</code>	<code>Key</code>
Virtual scroll (CDK)	<code>ListView.builder</code> (lazy sẵn)
Web Worker	<code>Isolate (compute)</code>
Chrome DevTools Performance	Flutter DevTools Performance + CPU profiler
<code>ng build --prod</code> rồi đo	<code>flutter run --profile</code> rồi đo

Ví dụ

const làm Flutter bỏ qua việc build lại

`examples/lib/chapters/ch02/rebuild_demo.dart`:

```

class _RebuildDemoState extends State<RebuildDemo> {
  int _ticks = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Text('Đã bấm $_ticks lần'),
        const BuildCounter(label: 'const'), // cùng một đối tượng mỗi lần → Flutter bỏ
        qua, không build lại
        // ignore: prefer_const_constructors
        BuildCounter(label: 'không const'), // đối tượng mới mỗi lần → build lại
        FilledButton(onPressed: () => setState(() => _ticks++), child: const
Text('setState')),
      ],
    );
  }
}

```

`BuildCounter` đếm số lần `build` của nó. Test bấm 2 lần:

```
expect(BuildCounter.counts, {'const': 1, 'không const': 3});
```

Widget `const` là **cùng một đối tượng** ở mọi lần `build` của cha → Flutter so sánh thấy giống hệt → không `build` lại con. Lint `prefer_const_constructors` (trong `flutter_lints`) nhắc bạn thêm `const`; dòng `// ignore: prefer_const_constructors` ở trên chỉ để minh họa.

AnimatedBuilder **VỚI** child

Docs liệt kê đây: đặt cây con **không phụ thuộc animation** bên trong `builder` → `build` lại mỗi tick. Cách đúng (`examples/lib/chapters/ch02/exercise_solution.dart`):

```

return AnimatedBuilder(
  animation: _controller,
  // child được build MỘT lần, rồi truyền lại vào builder ở mọi frame.
  child: const BuildCounter(label: 'logo'),
  builder: (context, child) => Transform.rotate(angle: _controller.value * 6.283,
child: child),
);

```

Test cho animation chạy 10 frame → `BuildCounter.counts['logo'] == 1`.

Kết quả thật (2026-09-30, phần Ch.2 của `test/chapters_test.dart`):

```

00:00 +3: Ch.2 – performance const không build lại; không const thì build lại
00:01 +4: Ch.2 – performance Bài 1: child của AnimatedBuilder chỉ build một lần

```

Danh sách và isolate (nhắc lại Tập 1-2)

- `ListView.builder` + `itemExtent` (Tập 1, Ch.6); tránh `shrinkWrap: true` với list dài — mất tính lazy.
- Parse JSON lớn / lọc hàng nghìn phần tử: `compute(parse, data)` (Tập 2, Ch.2).
- Tìm kiếm trong SQLite (có chỉ mục) nhanh hơn lọc list trong Dart khi dữ liệu lớn (Tập 2, Ch.4).

Đi sâu

Đo đúng cách: profile mode + DevTools

DOCS `testing/build-modes`: **debug** chậm (JIT, nhiều kiểm tra) — **không** dùng để đo; **profile** gần giống release nhưng vẫn cho DevTools kết nối; **release** để phát hành.

```
flutter run --profile           # trên máy thật (iPhone cần Mac + Xcode); không chạy
profile trên máy ảo iOS
```

Trong DevTools:

- **Performance**: biểu đồ frame; frame đỏ là vượt ngân sách. Xem "UI" (build) hay "Raster" (vẽ) chậm.
- **CPU profiler**: hàm nào tốn thời gian.
- **Memory**: rò rỉ (ví dụ controller không dispose).
- **Widget rebuild stats** (plugin IDE, mục "Show widget rebuild information"): widget nào build lại nhiều.

NOT RUN trong sandbox (không có thiết bị để đo profile mode).

Những thứ tốn kém (theo docs)

- `saveLayer` (gây ra bởi một số `Opacity`, `ShaderMask`, `ColorFilter`, clip có anti-alias...): tốn bộ nhớ GPU. Dùng `AnimatedOpacity`/`FadeTransition` thay vì `Opacity` trong animation.
- **Intrinsic pass** (`IntrinsicHeight`, `IntrinsicWidth`): bắt đo con hai lần; tránh trong list.
- Xây chuỗi dài bằng `+=` trong vòng lặp → dùng `StringBuffer`.

Kích thước app

`flutter build apk --analyze-size` / `flutter build ios --analyze-size` tạo báo cáo xem trong DevTools (App Size). Ảnh lớn, font thừa, package nặng là nguyên nhân hay gặp. Icon Material được **tree-shake** tự động khi build (log build web của sách: "Font asset MaterialIcons-Regular.otf was tree-shaken, reducing it from 1645184 to 7736 bytes (99.5% reduction)").

Web

Docs có trang "Web performance". Bản web của sách build bằng `--release` (dart2js, tối ưu). Thử `--wasm` cho trình duyệt hỗ trợ WasmGC (Chrome), **nhưng không dùng cho iPhone** (Tập 1, Ch.1).

Lỗi và bẫy thường gặp

- **Đo ở debug mode** → kết luận sai.
- **Tối ưu trước khi đo.**
- `setState` ở widget gốc cho một thay đổi nhỏ → cả cây build lại. Đưa state xuống thấp, hoặc dùng `select`.
- **Tạo đối tượng nặng trong build** (RegExp, định dạng ngày, list lớn) → tạo một lần ở field `final`.
- **Animation dùng opacity** thay vì `FadeTransition`.
- **Ảnh mạng không cache, kích thước gốc lớn** → dùng `cacheWidth/cacheHeight` khi giải mã.

Tóm tắt

- Ngân sách ~16 ms/frame (60 Hz), ~8 ms (120 Hz). Đo ở profile mode bằng DevTools.
- `const`, widget nhỏ, `select`, `child` của `AnimatedBuilder` giảm số lần build — test đếm build chứng minh được.
- List lazy, isolate cho việc nặng, tránh `saveLayer/intrinsic` không cần thiết.

Bài tập (có lời giải)

Bài 1. Một logo xoay dùng `AnimatedBuilder` nhưng đặt `const BuildCounter(label: 'logo')` trong `builder`. Sửa để logo chỉ build một lần, và viết test chứng minh.

▼ Lời giải

Chuyển logo vào tham số `child` (code `SpinningLogo` ở trên). Test:

```
testWidgets('Bài 1: child của AnimatedBuilder chỉ build một lần', (tester) async {
  BuildCounter.counts.clear();
  await pumpApp(tester, const SpinningLogo());
  for (var i = 0; i < 10; i++) {
    await tester.pump(const Duration(milliseconds: 16));
  }
  expect(BuildCounter.counts['logo'], 1);
  await tester.pumpWidget(const SizedBox()); // gỡ để dừng animation lặp
});
```


Lưu ý: vì `BuildCounter` là `const`, ngay cả khi đặt trong `builder` nó cũng **không** build lại (cùng đối tượng). Bài học thật sự quan trọng khi cây con **không** `const` được (ví dụ phụ thuộc dữ liệu) — khi đó chỉ tham số `child` mới cứu được.

Bài 2. Màn hình danh sách từ (Tập 2) dùng `context.watch<WordListViewModel>()` ở đầu `build`. Khi chỉ `loading` đổi, cả màn hình build lại. Đề xuất cách giảm.

▼ Lời giải

Tách thanh tiến độ thành widget riêng dùng `context.select<WordListViewModel, bool>((vm) => vm.loading)`; danh sách dùng `select((vm) => vm.words)` (list mới mỗi lần tải nên so sánh tham chiếu là đủ). Như vậy đổi `loading` chỉ build lại thanh tiến độ. Kỹ thuật `select` đã được chứng minh bằng test đếm build ở Tập 2, Chương 1 (Bài 1). Với màn hình vài chục dòng thì không cần — **đo trước** bằng DevTools rồi mới tách.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Performance best practices — <https://docs.flutter.dev/perf/best-practices> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/best-practices.md>
- Flutter performance profiling — <https://docs.flutter.dev/perf/ui-performance> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/ui-performance.md>
- Build modes — <https://docs.flutter.dev/testing/build-modes> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/build-modes.md>
- Measuring your app's size — <https://docs.flutter.dev/perf/app-size> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/app-size.md>
- DevTools Performance view — <https://docs.flutter.dev/tools/devtools/performance> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/tools/devtools/performance.md>
- Concurrency and isolates — <https://docs.flutter.dev/perf/isolates> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/isolates.md>
- Web performance — <https://docs.flutter.dev/perf/web-performance> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/perf/web-performance.md>

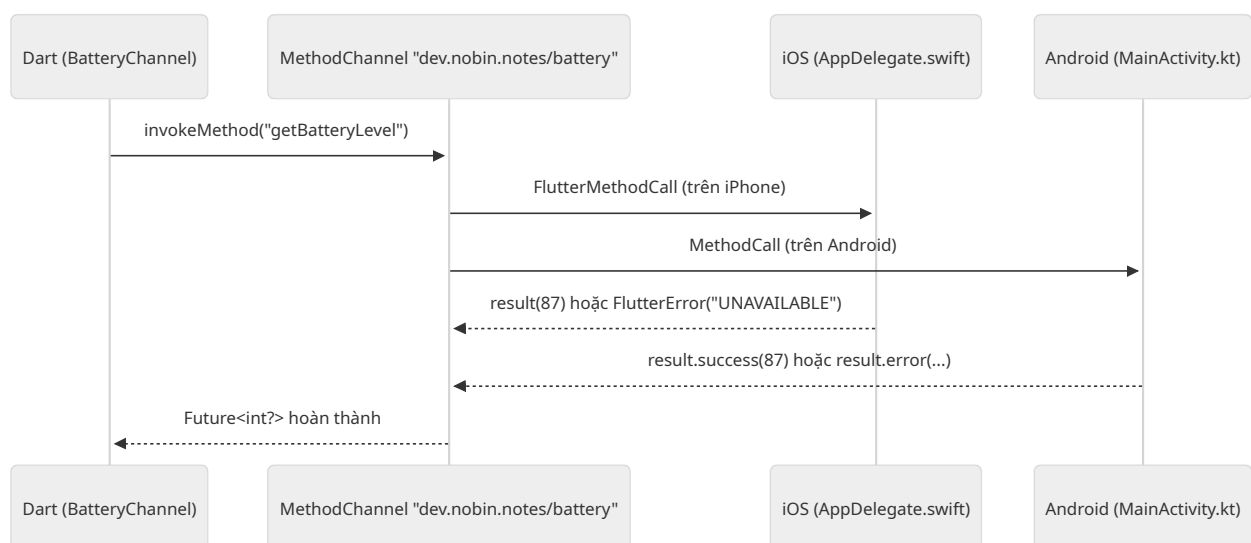
Chương 3 — Platform channels và plugin: gọi Swift/Kotlin từ Dart

Mục tiêu

- Hiểu **platform channel**: Dart gửi thông điệp bất đồng bộ tới code native (Swift trên iOS, Kotlin trên Android) và nhận kết quả.
- Viết một **MethodChannel** hoàn chỉnh (theo ví dụ "battery" của docs): Dart + Swift + Kotlin.
- Xử lý lỗi: `PlatformException`, `MissingPluginException` (web / chưa có code native).
- Test phía Dart bằng `setMockMethodCallHandler` — không cần thiết bị.
- Biết khi nào dùng `EventChannel`, **Pigeon**, và khi nào tách thành **plugin** riêng; biết Swift Package Manager.

Giải thích đơn giản

Docs "Writing custom platform-specific code": phần Flutter (client) gửi thông điệp qua một **kênh có tên**; phần native (host) lắng nghe kênh đó, gọi API của hệ điều hành, rồi trả lời. Thông điệp và trả lời đi **bất đồng bộ** để UI không bị chặn.



React Native (sách RN, Tập 3)	Flutter
TurboModule / Expo Modules API	MethodChannel / Pigeon / plugin
Codegen từ TypeScript spec	Pigeon sinh code từ Dart
Cần development build (không chạy trong Expo Go)	Chạy ngay khi <code>flutter run</code> build lại app

Ví dụ

Phía Dart — examples/lib/core/platform/battery_channel.dart

```
class BatteryChannel {
  const BatteryChannel([this._channel = const MethodChannel(channelName)]);

  static const channelName = 'dev.nobin.notes/battery';
  final MethodChannel _channel;

  /// Phần trăm pin, hoặc `null` nếu nền tảng không hỗ trợ (web, máy ảo không có pin...).
  Future<int?> batteryLevel() async {
    try {
      return await _channel.invokeMethod<int>('getBatteryLevel');
    } on PlatformException {
      return null; // native báo lỗi, ví dụ UNAVAILABLE
    } on MissingPluginException {
      return null; // không có code native cho kênh này (web, test)
    }
  }
}
```

Tên kênh nên có tiền tố duy nhất (domain ngược) để không trùng plugin khác.

Phía iOS — examples/ios/Runner/AppDelegate.swift

```
@main
@objc class AppDelegate: FlutterAppDelegate, FlutterImplicitEngineDelegate {
    // Theo docs: tạo kênh trong didInitializeImplicitFlutterEngine
    // (app dùng UIWindow lifecycle, window có thể nil trong didFinishLaunching).
    func didInitializeImplicitFlutterEngine(_ engineBridge: FlutterImplicitEngineBridge)
{
    GeneratedPluginRegistrant.register(with: engineBridge.pluginRegistry)

    let channel = FlutterMethodChannel(
        name: "dev.nobin.notes/battery",
        binaryMessenger: engineBridge.applicationRegistrar.messenger()
    )
    channel.setMethodCallHandler { [weak self] (call: FlutterMethodCall, result:
@escaping FlutterResult) in
        switch call.method {
        case "getBatteryLevel":
            self?.receiveBatteryLevel(result: result)
        case "isLowPowerMode":
            result(ProcessInfo.processInfo.isLowPowerModeEnabled)
        default:
            result(FlutterMethodNotImplemented)
        }
    }

    private func receiveBatteryLevel(result: FlutterResult) {
        let device = UIDevice.current
        device.isBatteryMonitoringEnabled = true
        if device.batteryState == UIDevice.BatteryState.unknown {
            result(FlutterError(code: "UNAVAILABLE", message: "Battery level not
available.", details: nil))
        } else {
            result(Int(device.batteryLevel * 100))
        }
    }
}
```

Docs ghi chú: từ Flutter 3.41, app mặc định dùng vòng đời `UISceneDelegate`, nên `window` có thể là `nil` trong `application(_:didFinishLaunchingWithOptions:)` — tạo kênh trong `didInitializeImplicitFlutterEngine` để tránh crash. Template `flutter create` của Flutter 3.47.5 đã sinh sẵn protocol `FlutterImplicitEngineDelegate`.

```
class MainActivity : FlutterActivity() {
    private val channelName = "dev.nobin.notes/battery"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
channelName).setMethodCallHandler { call, result ->
            when (call.method) {
                "getBatteryLevel" -> {
                    val level = getBatteryLevel()
                    if (level != -1) result.success(level)
                    else result.error("UNAVAILABLE", "Battery level not available.",
null)
                }
                "isLowPowerMode" -> {
                    val pm = getSystemService(Context.POWER_SERVICE) as PowerManager
                    result.success(pm.isPowerSaveMode)
                }
                else -> result.notImplemented()
            }
        }

        private fun getBatteryLevel(): Int {
            val batteryManager = getSystemService(Context.BATTERY_SERVICE) as
BatteryManager
            return batteryManager.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY)
        }
    }
}
```

Biên dịch và chạy code Swift/Kotlin này: NOT RUN (không có Xcode / Android SDK trong sandbox). Code theo sát mẫu của docs.

Test phía Dart — giả lập native

```
const channel = MethodChannel(BatteryChannel.channelName);
final messenger = TestDefaultBinaryMessengerBinding.instance.defaultBinaryMessenger;
tearDown(() => messenger.setMockMethodCallHandler(channel, null));

test('getBatteryLevel trả 87', () async {
  final calls = <String>[];
  messenger.setMockMethodCallHandler(channel, (call) async {
    calls.add(call.method);
    return call.method == 'getBatteryLevel' ? 87 : null;
  });
  expect(await const BatteryChannel().batteryLevel(), 87);
  expect(calls, ['getBatteryLevel']);
});

test('native báo lỗi UNAVAILABLE → null', () async {
  messenger.setMockMethodCallHandler(channel, (call) async => throw
PlatformException(code: 'UNAVAILABLE'));
  expect(await const BatteryChannel().batteryLevel(), isNull);
});
```

Kết quả thật (2026-09-30, flutter test --reporter expanded test/platform_channel_test.dart):

```
00:00 +0: getBatteryLevel trả 87
00:00 +1: native báo lỗi UNAVAILABLE → null
00:00 +2: không có code native (web, chưa đăng ký) → null
00:00 +3: Bài tập: isLowPowerMode
00:00 +4: All tests passed!
```

Trên **web**, kênh không có code native → `MissingPluginException` → màn hình Cài đặt hiện "Không hỗ trợ trên nền tảng này" (bản build web đã chạy trong Chromium headless, xem logs/).

Đi sâu

Bẫy thật: kênh chưa mock trong widget test có thể treo

Khi viết `app_test.dart`, sách thấy trong `testWidgets` lời gọi tới kênh **chưa được mock** không bao giờ hoàn thành (`pump`, `pumpAndSettle`, `runAsync` đều không giúp) — trong khi ở `test()` thường thì nó ném `MissingPluginException` ngay. Cách làm chắc chắn: **luôn mock** mọi kênh mà widget test chạm tới:

```
messenger.setMockMethodCallHandler(channel, (call) async => throw
MissingPluginException());
addTearDown(() => messenger.setMockMethodCallHandler(channel, null));
```

Kiểu dữ liệu đi qua kênh

`StandardMessageCodec` hỗ trợ: `null`, `bool`, `int`, `double`, `String`, `Uint8List`..., `List`, `Map`. Muốn gửi object → chuyển thành `Map`. Để sai tên khóa, sai kiểu → dùng **Pigeon**.

Pigeon — kênh có kiểu (type-safe)

Pigeon (package của team Flutter) đọc một file Dart mô tả API và **sinh code** Dart + Swift + Kotlin khớp nhau — không còn chuỗi tên hàm "gỗ tay". Docs khuyên dùng khi API native có nhiều hàm. Sách chỉ giới thiệu (chưa thêm vào dự án).

EventChannel — luồng sự kiện từ native

Khi native cần **đẩy** dữ liệu liên tục (cảm biến, trạng thái pin thay đổi), dùng `EventChannel` → phía Dart nhận một `Stream`.

Khi nào tách thành plugin?

Code native dùng lại ở nhiều app → tạo **plugin package** (`flutter create --template=plugin`), có `ios/`, `android/`, `web/` riêng. Từ Flutter 3.44, plugin iOS nên hỗ trợ **Swift Package Manager** (docs `swift-package-manager/for-plugin-authors`); Flutter vẫn dùng CocoaPods cho plugin chưa có SwiftPM, nhưng registry CocoaPods sẽ chỉ-đọc từ 2026-12-02.

Luồng (thread)

Handler phía native chạy trên **main thread**. Việc nặng (đọc file lớn) → chuyển sang luồng nền trong Swift/Kotlin rồi gọi `result` (docs có mục riêng về threading của platform channel).

Lỗi và bẫy thường gặp

- Sai tên kênh / tên hàm giữa Dart và native → `MissingPluginException` / `notImplemented`.
- Tạo kênh iOS trong `didFinishLaunching` với `UIScene` → crash vì `window` nil (docs).
- Quên xử lý `MissingPluginException` → crash trên web.
- Gọi `result` hai lần (hoặc không gọi) ở native → lỗi / Future treo.
- Widget test chạm kênh chưa mock → treo (xem trên).
- Hot reload sau khi sửa Swift/Kotlin → không áp dụng; phải build lại.

Tóm tắt

- MethodChannel: Dart `invokeMethod` ↔ native `setMethodCallHandler`; bắt đồng bộ; lỗi qua `PlatformException`.
- iOS: tạo kênh trong `didInitializeImplicitFlutterEngine`; Android: trong `configureFlutterEngine`.

- Test phía Dart bằng `getMockMethodCallHandler`. API lớn → Pigeon. Dừng lại → plugin (hỗ trợ SwiftPM).

Bài tập (có lời giải)

Bài 1. Thêm hàm `lowPowerMode()` (iOS: Low Power Mode, Android: Battery Saver) vào `BatteryChannel` và test phía Dart.

▼ Lời giải

Dart:

```
Future<bool?> lowPowerMode() async {
  try {
    return await _channel.invokeMethod<bool>('isLowPowerMode');
  } on PlatformException {
    return null;
  } on MissingPluginException {
    return null;
  }
}
```

Native: Swift `case "isLowPowerMode": result(ProcessInfo.processInfo.isLowPowerModeEnabled);`

Kotlin `"isLowPowerMode" -> result.success((getSystemService(Context.POWER_SERVICE) as PowerManager).isPowerSaveMode)` (đã có trong hai file ở trên). Test: mock trả `true` cho `isLowPowerMode` → `lowPowerMode()` là `true` (test "Bài tập: isLowPowerMode" pass). Phía native: NOT RUN.

Bài 2. Vì sao `BatteryChannel` nhận `MethodChannel` qua constructor (có giá trị mặc định), và vì sao nó được cung cấp qua provider?

▼ Lời giải

(1) Có thể truyền kênh khác (tên khác cho flavor, hoặc một kênh giả) mà không sửa class. (2) Qua provider, màn hình Cài đặt và Lab lấy `context.read<BatteryChannel>()` — test có thể thay bằng lớp giả `implements BatteryChannel` nếu muốn, hoặc giữ lớp thật và mock ở tầng messenger (cách sách dùng). Đây vẫn là tinh thần "DI + fake" của docs kiến trúc, áp dụng cho code native.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Writing custom platform-specific code — <https://docs.flutter.dev/platform-integration/platform-channels> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/platform-channels.md>
- Developing packages & plugins — <https://docs.flutter.dev/packages-and-plugins/developing-packages> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/packages-and-plugins/developing-packages.md>
- Swift Package Manager for plugin authors — <https://docs.flutter.dev/packages-and-plugins/swift-package-manager/for-plugin-authors> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/packages-and-plugins/swift-package-manager/for-plugin-authors.md>
- Mock platform channels in tests (breaking change note) — <https://docs.flutter.dev/release/breaking-changes/mock-platform-channels> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/release/breaking-changes/mock-platform-channels.md>
- Package pigeon (29.0.4, kiểm tra 2026-09-30): <https://pub.dev/packages/pigeon>
- Sách React Native trong repo: Tập 3, Chương 2 — Native modules

Chương 4 — Bảo mật: lưu bí mật, PIN, khóa tự động, deep link, obfuscation

Mục tiêu

- Lưu bí mật đúng chỗ: **Keychain** / **Keystore** qua `flutter_secure_storage` — không phải `shared_preferences`.
- Bấm PIN có muối (salt), so sánh constant-time, **khóa tạm** khi nhập sai nhiều lần, từ chối PIN yếu.
- **Tự khóa** khi app vào nền quá lâu (`AppLifecycleListener`) và chặn mọi màn hình bằng `redirect` của `go_router`.
- Kiểm tra dữ liệu từ bên ngoài (deep link) và **không log dữ liệu nhạy cảm**.
- Biết obfuscation làm được gì và **không** làm được gì; biết lời khuyên bảo mật của team Flutter.

Giải thích đơn giản

Nguyên tắc: **app chạy trên máy của người khác** — ai có máy (hoặc file cài đặt) đều có thể đọc code và dữ liệu nếu không được bảo vệ. Docs "Obfuscate Dart code" cảnh báo thẳng: lưu bí mật (API key của server...) **trong app** là thực hành kém; obfuscation chỉ đổi tên biểu tượng, **không** mã hóa tài nguyên và **không** chống dịch ngược.

Dữ liệu	Để ở đâu	Vì sao
Cài đặt (theme)	<code>shared_preferences</code>	Không nhạy cảm
Token đăng nhập, PIN (đã bấm), ghi chú bí mật	<code>flutter_secure_storage</code> (Keychain/Keystore)	Được hệ điều hành mã hóa, gắn với thiết bị
API key bí mật của server	Không để trong app — để trên server	Ai cũng lấy được từ file cài đặt

`flutter_secure_storage` 11.2.0 (README): iOS dùng **Keychain**; Android mã hóa bằng khóa trong **Keystore** (mặc định RSA-OAEP bọc khóa + AES-GCM); web dùng WebCrypto và **chỉ** chạy trên **HTTPS hoặc localhost**.

Ví dụ

Kho bí mật — examples/lib/core/security/secure_store.dart

```
abstract interface class SecureStore {
  Future<String?> read(String key);
  Future<void> write(String key, String value);
  Future<void> delete(String key);
}

class FlutterSecureStore implements SecureStore {
  FlutterSecureStore([FlutterSecureStorage? storage])
    : _storage =
        storage ??
        const FlutterSecureStorage(
          // Chỉ đọc được sau khi máy mở khóa lần đầu kể từ lúc khởi động (README:
          IOSOptions.accessibility).
          iOptions: IOSOptions(accessibility:
            KeychainAccessibility.first_unlock_this_device),
        );
  // read / write / delete gọi _storage
}
```

`first_unlock_this_device`: không đọc được trước lần mở khóa đầu tiên sau khi khởi động, và **không** chuyển sang máy khác qua bản sao lưu. Android: README khuyên tắt auto backup (`android:allowBackup="false"`) để tránh lỗi khóa khi khôi phục — dự án đã đặt.

Băm PIN — examples/lib/features/auth/domain/pin_hasher.dart

```
/// Muối (salt) ngẫu nhiên 16 byte từ bộ sinh số an toàn (Random.secure) – mỗi PIN một muối.
String generateSalt([Random? random]) {
  final r = random ?? Random.secure();
  return base64Encode(List<int>.generate(16, (_) => r.nextInt(256)));
}

/// Băm PIN = SHA-256 lặp [iterations] lần với muối. KHÔNG lưu PIN gốc.
String hashPin(String pin, String salt, {int iterations = 10000}) {
  final saltBytes = base64Decode(salt);
  var digest = sha256.convert([...saltBytes, ...utf8.encode(pin)]).bytes;
  for (var i = 1; i < iterations; i++) {
    digest = sha256.convert([...digest, ...saltBytes]).bytes;
  }
  return base64Encode(digest);
}

/// So sánh không để lộ thời gian (constant-time): luôn duyệt hết chuỗi.
bool constantTimeEquals(String a, String b) {
  if (a.length != b.length) return false;
  var diff = 0;
  for (var i = 0; i < a.length; i++) {
    diff |= a.codeUnitAt(i) ^ b.codeUnitAt(i);
  }
  return diff == 0;
}

/// Thời gian khóa sau [failedAttempts] lần sai liên tiếp: 5 lần → 30 giây, mỗi lần sai thêm gấp đôi, tối đa 1 giờ.
Duration lockoutDuration(int failedAttempts) {
  if (failedAttempts < 5) return Duration.zero;
  final seconds = 30 * pow(2, failedAttempts - 5).toInt();
  return Duration(seconds: min(seconds, 3600));
}
```

Nói thật về giới hạn: PIN 4–6 số chỉ có 10^4 – 10^6 khả năng. Nếu kẻ xấu lấy được muối + hash, họ vẫn dò ra PIN (dù lặp SHA-256 làm chậm hơn). Lớp bảo vệ thật sự là: hash nằm trong **Keychain/Keystore** + **khóa tạm** sau nhiều lần sai + tự khóa. Thuật toán chuyên cho mật khẩu (PBKDF2, scrypt, Argon2) tốt hơn SHA-256 lặp; package `crypto` không có sẵn chúng — xem "Ideas for later".

`PinRepository.verify` trả về sealed class `PinOk` / `PinWrong(failedAttempts)` / `PinLocked(remaining)` và lưu bộ đếm sai + thời điểm hết khóa **trong SecureStore** (không trong bộ nhớ — đóng app mở lại không xóa được bộ đếm).

Tự khóa + chặn màn hình

```
// app.dart - theo dõi vòng đời app
_lifecycle = AppLifecycleListener(onHide: _auth.onBackground, onShow: _auth.onResume);

// router.dart - guard
redirect: (context, state) {
  final atLock = state.matchedLocation == '/lock';
  final unlocked = auth.status == AuthStatus.unlocked;
  if (!unlocked) return atLock ? null : '/lock';
  if (atLock) return '/notes';
  return null;
},
refreshListenable: auth,
```

`AuthController.onResume` khóa lại nếu app ở nền lâu hơn `autoLockAfter` (30 giây, chỉnh trong Cài đặt). Test:

```
auth.onBackground();
now = now.add(const Duration(minutes: 2));
auth.onResume();
expect(auth.status, AuthStatus.locked);
```

Deep link — kiểm tra trước khi dùng

```
String? parseNoteDeepLink(Uri uri) {
  final idRe = RegExp(r'^[a-z0-9-]{1,40}$');
  final segments = switch (uri) {
    Uri(scheme: 'sochichu', host: 'notes') => uri.pathSegments,
    Uri(scheme: 'https', host: 'nobin.dev') when uri.pathSegments.firstOrNull ==
'notes' =>
      uri.pathSegments.skip(1).toList(),
    _ => const <String>[],
  };
  if (segments.length != 1) return null;
  final id = segments.single;
  return idRe.hasMatch(id) ? id : null;
}
```

Phát hiện khi test: `Uri.parse('sochichu://notes/../../../../etc')` được **chuẩn hóa** thành path `/etc` → hàm trả `'etc'`. Vẫn an toàn vì id chỉ dùng để tra cứu (không ghép thành đường dẫn file); bài học: **kiểm tra trên giá trị sau cùng**, sau khi parse. (Sách React Native trong repo gặp đúng hiện tượng này với `URL()` của JavaScript.)

Không log dữ liệu nhạy cảm

`AppLogger` luôn chạy `redact()` trước khi lưu/gửi: email → `<email>`, dãy từ 4 chữ số → `<số>`. `AuthController` chỉ log "Đã tạo PIN", không bao giờ log giá trị. Test toàn app kiểm tra không bản ghi nào chứa "2468".

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/auth_test.dart`):

```
00:00 +0: pin_hasher (domain thuần) muối khác nhau → hash khác nhau; cùng muối → cùng hash; không chứa PIN gốc
00:00 +1: pin_hasher (domain thuần) constantTimeEquals, isValidPinFormat
00:00 +2: pin_hasher (domain thuần) lockoutDuration: 5 lần → 30s, gấp đôi, tối đa 1 giờ
00:00 +3: pin_hasher (domain thuần) shouldAutoLock
00:00 +4: PinRepository chỉ lưu muối + hash; sai 5 lần thì khóa tạm; hết giờ khóa thì mở được
00:00 +5: AuthController tạo PIN: từ chối PIN yếu và PIN không khớp
00:00 +6: AuthController khóa, mở sai, mở đúng; tự khóa khi ở nền quá lâu
00:00 +7: All tests passed!
```

Đi sâu

Obfuscation

```
flutter build apk --obfuscate --split-debug-info=build/symbols
flutter build ipa --obfuscate --split-debug-info=build/symbols
```

Docs: chỉ có hiệu lực ở **release**; `--obfuscate` luôn đi cùng `--split-debug-info` (file ký hiệu để giải mã stack trace bằng `flutter symbolize` — **phải lưu lại** file này cho từng bản phát hành); web không obfuscate mà được minify. Code dựa vào tên kiểu (`runtimeType.toString()`, `Enum.toString()`) sẽ ra tên lạ sau khi obfuscate — app của sách dùng `enum.name`/chuỗi cố định.

Lời khuyên của team Flutter (trang Security)

- Luôn dùng **Flutter stable mới nhất** và **cập nhật package** (bản vá bảo mật).
- Docs khuyên **tránh ghim cứng phiên bản** package; nếu ghim thì phải kiểm tra định kỳ. **Sách ghim exact** để mọi người (và app TOEIC — Task 9) build ra cùng kết quả; đổi lại, hãy chạy `flutter pub outdated` hằng tháng và nâng phiên bản có chủ đích (xem STACK.md). Đây là đánh đổi có ý thức, không phải bỏ qua lời khuyên.

Thêm nếu app có server

HTTPS bắt buộc (iOS chặn HTTP mặc định); token ngắn hạn + refresh; kiểm tra quyền ở **server**; cân nhắc certificate pinning cho app nhạy cảm (có rủi ro khi chứng chỉ đổi).

Sinh trắc học (Face ID)

Package `local_auth` (3.0.2, pub.dev 2026-09-30) cho phép mở khóa bằng Face ID/vân tay; `flutter_secure_storage` CÓ `AndroidOptions.biometric`. Sách không dùng trong app mẫu (cần thiết bị thật để kiểm tra) — "Ideas for later".

Lỗi và bẫy thường gặp

- Lưu token/PIN trong `shared_preferences` (dạng rõ) → ai có file sao lưu là đọc được.
- Lưu PIN gốc hoặc băm không muối → dò bảng (rainbow table).
- So sánh hash bằng `==` → về lý thuyết lộ thời gian; dùng constant-time.
- Bộ đếm sai chỉ trong RAM → tắt app mở lại là hết khóa.
- Tin deep link → mở màn hình / tham số không mong muốn; luôn kiểm tra.
- Log dữ liệu nhạy cảm (PIN, token, nội dung ghi chú) → lộ qua crash report.
- Nhét API key bí mật vào app rồi tin obfuscation che được.

Tóm tắt

- Bí mật → Keychain/Keystore (`flutter_secure_storage`); cài đặt → `shared_preferences`; khóa server → server.
- PIN: muối + băm + constant-time + khóa tạm lưu bền + từ chối PIN yếu; tự khóa khi vào nền; guard bằng `redirect`.
- Kiểm tra deep link sau khi parse; `redact` log; obfuscation chỉ là lớp phụ.

Bài tập (có lời giải)

Bài 1. Viết `isWeakPin(pin)`: yếu nếu mọi chữ số giống nhau (0000), là dãy tăng/giảm liên tiếp (1234, 9876), hoặc nằm trong danh sách hay gặp. Dùng nó khi tạo PIN.

▼ Lời giải

`examples/lib/features/auth/domain/pin_hasher.dart` (file `chapters/ch04/exercise_solution.dart` export lại):

```
bool isWeakPin(String pin) {
  if (pin.isEmpty) return true;
  if (pin.split('').toSet().length == 1) return true; // 0000, 1111
  final digits = pin.codeUnits.map((c) => c - 48).toList();
  bool stepBy(int step) {
    for (var i = 1; i < digits.length; i++) {
      if (digits[i] - digits[i - 1] != step) return false;
    }
    return true;
  }

  if (stepBy(1) || stepBy(-1)) return true; // 1234, 9876
  const common = {'1212', '1004', '2000', '6969', '4321', '1122', '2580', '0852'};
  return common.contains(pin);
}
```

`AuthController.createPin` gọi `isWeakPin` và báo "PIN quá dễ đoán, hãy chọn PIN khác" (test toàn app nhập 1234 và thấy thông báo này). Danh sách "hay gặp" ở đây là ví dụ do sách chọn, không phải thống kê chính thức.

Bài 2. Vì sao bộ đếm số lần sai và thời điểm hết khóa được lưu trong `SecureStore` chứ không phải trong một biến của `AuthController`?

▼ Lời giải

Nếu chỉ trong RAM, kẻ xấu vuốt tắt app rồi mở lại là bộ đếm về 0 → thử PIN không giới hạn. Lưu trong Keychain/Keystore thì bộ đếm sống qua các lần mở app và khó bị sửa. Test "PinRepository ... sai 5 lần thì khóa tạm; hết giờ khóa thì mở được" chứng minh: lần sai thứ 5 trả `PinLocked(30 giây)`, nhập **đúng** PIN trong lúc khóa vẫn bị từ chối, sau 31 giây mới mở được.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Security — <https://docs.flutter.dev/security> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/security/index.md>
- Obfuscate Dart code — <https://docs.flutter.dev/deployment/obfuscate> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/obfuscate.md>
- Navigation and routing (redirect / deep link) — <https://docs.flutter.dev/ui/navigation> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/ui/navigation/index.md>
- flutter_secure_storage 11.2.0 (README: Keychain/Keystore, web chỉ HTTPS/localhost, tắt auto backup): https://pub.dev/packages/flutter_secure_storage
- crypto 3.0.7: <https://pub.dev/packages/crypto> · local_auth: https://pub.dev/packages/local_auth
- Sách React Native trong repo: [Tập 3, Chương 4 — Security](#)

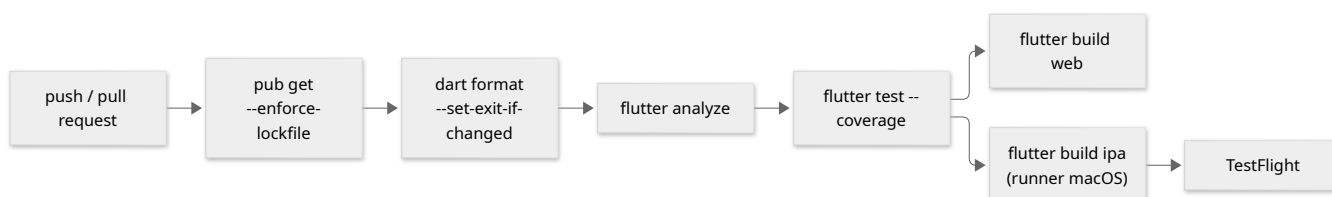
Chương 5 — CI/CD: tự động kiểm tra, build và giao app

Mục tiêu

- Hiểu CI (tích hợp liên tục) và CD (giao liên tục) cho app Flutter.
- Viết workflow **GitHub Actions**: pub get → format → analyze → test (coverage) → build web; build iOS + TestFlight trên runner macOS.
- Biết các lựa chọn docs liệt kê: **Codemagic**, **Bitrise** (có sẵn Flutter), và **fastlane** kết hợp CI có sẵn.
- Quản lý version/build number tự động, và giữ bí mật (chứng chỉ, khóa API) trong **secrets**.

Giải thích đơn giản

Mỗi lần đẩy code, máy CI chạy đúng những lệnh bạn chạy tay — nhờ đó lỗi được bắt **trước** khi tới người dùng. Với Flutter:



Docs "Continuous delivery with Flutter" chia hai hướng: (1) dịch vụ **all-in-one** có sẵn Flutter: Codemagic, Bitrise; (2) dùng **fastlane** cùng hệ thống CI có sẵn (GitHub Actions, GitLab, CircleCI...). **Build iOS luôn cần macOS** — GitHub Actions có runner `macos-latest`, nên Nobin không cần tự mua Mac chỉ để **build** (vẫn cần tài khoản Apple Developer để ký và phát hành).

Sách giữ `scripts/check-all.sh` làm "CI chạy tay" cho cả 3 tập — cùng các bước như workflow.

Ví dụ

Workflow kiểm tra — vol3-nang-cao/ci/flutter-ci.yml (file mẫu)

```
name: flutter-ci

on:
  push:
    branches: [main]
  pull_request:

jobs:
  check:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: books/flutter/vol3-nang-cao/examples
    steps:
      - uses: actions/checkout@v7
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          flutter-version: 3.47.5 # ghim đúng bản của sách (STACK.md)
          cache: true
      - run: flutter pub get --enforce-lockfile
      - run: dart format --output=none --set-exit-if-changed lib test
      - run: flutter analyze
      - run: flutter test --coverage
      - run: flutter build web --release --no-web-resources-cdn
      - uses: actions/upload-artifact@v7
        with:
          name: web-build
          path: books/flutter/vol3-nang-cao/examples/build/web
```

- `subosito/flutter-action` (MIT, tag mới nhất v2.23.0 kiểm tra bằng `git ls-remote` 2026-09-30) cài Flutter đúng phiên bản. `actions/checkout` và `actions/upload-artifact`: tag mới nhất v7.0.1.
- `--enforce-lockfile`: thất bại nếu `pubspec.lock` không khớp — CI build **đúng** phiên bản đã khóa.
- File này là **mẫu**, không đặt trong `.github/workflows/` của repo (ngoài phạm vi thư mục của sách). YAML được kiểm tra cú pháp trong `check-all.sh` ("yaml OK: 2 file"). **Chạy thật trên GitHub Actions: NOT RUN.**

Workflow iOS → TestFlight — ci/ios-testflight.yml (mẫu, NOT RUN)

```
jobs:
  ios:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v7
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          flutter-version: 3.47.5
      - run: flutter pub get --enforce-lockfile
      - run: flutter test
      - name: Build IPA
        run: >
          flutter build ipa --release
          --build-number=${{ github.run_number }}
          --obfuscate --split-debug-info=build/symbols
          --export-options-plist=ios/ExportOptions.plist
      - name: Upload to TestFlight
        run: |
          mkdir -p ~/private_keys
          echo "$ASC_PRIVATE_KEY" > ~/private_keys/AuthKey_${ASC_KEY_ID}.p8
          xcrun altool --upload-app --type ios -f build/ios/ipa/*.ipa --apiKey
"$ASC_KEY_ID" --apiIssuer "$ASC_ISSUER_ID"
```

- `flutter build ipa`, `--build-number`, `--export-options-plist`, và lệnh `xcrun altool --upload-app --type ios -f ... --apiKey ... --apiIssuer ...` theo docs "Build and release an iOS app". Việc altool tìm khóa trong `~/private_keys` là **UNVERIFIED**.
- Ký app (certificate + provisioning profile) trên CI: docs gợi ý **fastlane match**; hoặc để Xcode tự ký với App Store Connect API key. Cần **Apple Developer Program (\$99/năm)** — Chương 6.
- `flutter build ipa` **không có** trên Linux (sandbox: `flutter build -h` chỉ liệt kê aar, apk, appbundle, bundle, linux, web).

Tăng build number tự động — examples/lib/chapters/ch05/versioning.dart

```
/// (Flutter: phần trước "+" là version name (CFBundleShortVersionString /
versionName),
/// phần sau "+" là build number (CFBundleVersion / versionCode).)
String bumpBuildNumber(String version) {
  final m = RegExp(r'^(\d+\.\d+\.\d+)\+(\d+)$').firstMatch(version.trim());
  if (m == null) throw FormatException('Version không đúng dạng x.y.z+n: $version');
  return '${m.group(1)}+${int.parse(m.group(2)) + 1}';
}
```

Docs iOS/Android: `version: 1.0.0+1` trong `pubspec.yaml` → iOS `CFBundleShortVersionString` / `CFBundleVersion`, Android `versionName` / `versionCode`; có thể ghi đè khi build bằng `--build-name` / `--build-number`. Store yêu cầu build number **tăng dần** cho mỗi lần tải lên — CI dùng

`github.run_number` là cách đơn giản.

Kết quả thật (phần Ch.5–6 của `test/chapters_test.dart`, 2026-09-30):

```
00:01 +7: Ch.5-6 – version bumpBuildNumber và bumpVersion
00:01 +8: Ch.5-6 – version compareVersions / needsForceUpdate
```

Đi sâu

Bí mật trên CI

Chứng chỉ `.p12`, provisioning profile, khóa App Store Connect `.p8`, keystore Android `.jks` + `key.properties` → lưu trong **GitHub Secrets** (hoặc kho bí mật của dịch vụ CI), giải mã ra file tạm khi chạy, **không commit**. `.gitignore` của sách đã chặn `*.jks`, `*.keystore`, `key.properties`.

Cache và thời gian build

`flutter-action` CÓ `cache: true` (cache SDK). Runner macOS đắt và chậm hơn Linux: chạy `test/analyze` trên Linux mỗi PR; build iOS chỉ khi gắn tag hoặc bấm tay (`workflow_dispatch`).

Flavor / môi trường

Dev / staging / prod khác nhau ở API URL, tên app, icon → docs "Flavors" (Android product flavors, iOS schemes) + `--flavor`, hoặc `--dart-define=API_URL=...` cho giá trị đơn giản. Nhớ: `--dart-define` **không** phải chỗ giấu bí mật (vẫn nằm trong binary).

Codemagic / Bitrise / Xcode Cloud

Dịch vụ có sẵn bước Flutter, ký iOS, đẩy TestFlight bằng giao diện. Giá và hạn mức miễn phí: **UNVERIFIED** (trang của các dịch vụ bị chặn trong sandbox). Với một người học, GitHub Actions + fastlane đủ dùng.

Lỗi và bẫy thường gặp

- **CI dùng Flutter khác máy bạn** → lỗi khó hiểu. Ghim `flutter-version`.
- **Không** `--enforce-lockfile` → CI lạng lẽ lấy phiên bản package khác.
- **Commit keystore / .p8** → lộ khóa ký; phải thu hồi và tạo lại.
- **Build number không tăng** → App Store Connect / Play Console từ chối bản tải lên.
- **Mất file** `--split-debug-info` của bản đã phát hành → không đọc được stack trace crash (Chương 7).
- **Chạy build iOS trên runner Linux** → không có `flutter build ipa`.

Tóm tắt

- CI: pub get (lockfile) → format → analyze → test → build. Sách có `scripts/check-all.sh` và workflow mẫu `ci/*.yaml`.
- iOS cần macOS: dùng runner `macos-latest`; ký bằng fastlane match hoặc API key; cần Apple Developer Program.
- Build number tăng tự động; bí mật trong CI secrets.

Bài tập (có lời giải)

Bài 1. Viết `bumpVersion(version, part: 'major'|'minor'|'patch')` theo SemVer, đặt lại build number về 1.

▼ Lời giải

`examples/lib/chapters/ch05/versioning.dart`:

```
String bumpVersion(String version, {required String part}) {
  final m = RegExp(r'^(\d+)\.(\d+)\.(\d+)(\+\d+)?$').firstMatch(version.trim());
  if (m == null) throw FormatException('Version không đúng dạng: $version');
  var (major, minor, patch) = (int.parse(m.group(1)!), int.parse(m.group(2)!),
    int.parse(m.group(3)!));
  switch (part) {
    case 'major':
      (major, minor, patch) = (major + 1, 0, 0);
    case 'minor':
      (minor, patch) = (minor + 1, 0);
    case 'patch':
      patch++;
    default:
      throw ArgumentError('part phải là major/minor/patch');
  }
  return '$major.$minor.$patch+1';
}
```

Record + pattern gán nhiều biến một lúc: `(major, minor, patch) = (major + 1, 0, 0)`. Test: `1.2.3+9` → patch `1.2.4+1`, minor `1.3.0+1`; `1.2.3` → major `2.0.0+1`. (Lưu ý: nếu store yêu cầu build number tăng **toàn cục**, đừng đặt lại về 1 — dùng số lần chạy CI.)

Bài 2. Sửa `flutter-ci.yaml` để job build web **chỉ** chạy khi test pass, và chỉ trên nhánh `main`.

▼ Lời giải

Tách thành 2 job, job sau `needs` job trước và có điều kiện:

```
jobs:
  check:
    runs-on: ubuntu-latest
    steps: [ ... pub get, format, analyze, test ... ]
  build-web:
    needs: check
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps: [ ... checkout, flutter-action, pub get, flutter build web ... ]
```

Trong một job, các bước vốn đã dừng khi một bước lỗi; tách job giúp build chỉ chạy trên `main` và cho phép chạy song song các kiểm tra khác. Lời giải tham khảo: cú pháp `needs/ if` là cú pháp chuẩn của GitHub Actions nhưng workflow này **chưa chạy thật** (NOT RUN).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Continuous delivery with Flutter — <https://docs.flutter.dev/deployment/cd> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/cd.md>
- Build and release an iOS app — <https://docs.flutter.dev/deployment/ios> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/ios.md>
- Build and release an Android app — <https://docs.flutter.dev/deployment/android> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/android.md>
- Flavors — <https://docs.flutter.dev/deployment/flavors> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/flavors.md>
- subosito/flutter-action README (MIT): <https://github.com/subosito/flutter-action>
- actions/checkout: <https://github.com/actions/checkout> · actions/upload-artifact: <https://github.com/actions/upload-artifact>
- Sách React Native trong repo: Tập 3, Chương 5 — CI/CD

Chương 6 — Phát hành App Store và Google Play

Mục tiêu

- Biết đầy đủ các bước đưa app Flutter lên **App Store** (qua TestFlight) và **Google Play**, theo docs chính thức.
- Hiểu version name / build number, icon, launch screen, ký app (signing).
- Build bản phát hành: `flutter build ipa`, `flutter build appbundle`; obfuscation + file ký hiệu.
- Viết logic "bắt buộc cập nhật" (force update) bằng so sánh version.
- Có checklist phát hành cho app TOEIC.

Giải thích đơn giản

Bước	iOS (App Store)	Android (Google Play)
Tài khoản	Apple Developer Program (\$99/năm — developer.apple.com/programs , 2026-09-30)	Google Play Console (phí đăng ký: UNVERIFIED , trang Google bị chặn)
Định danh app	Bundle ID (<code>dev.nobin.tap3_so_ghi_chu</code>) — đăng ký trên developer.apple.com	Application ID (<code>applicationId</code> trong <code>build.gradle.kts</code>)
Tạo app trên store	App Store Connect → New App	Play Console → Create app
Ký app	Certificate + provisioning profile (Xcode tự ký được)	Upload keystore (<code>.jks</code>) + <code>key.properties</code>
Build	<code>flutter build ipa</code> (cần macOS + Xcode)	<code>flutter build appbundle</code> (định dạng Play khuyên)
Thử nội bộ	TestFlight	Internal testing track
Phát hành	Gửi xét duyệt (App Review)	Gửi xét duyệt, phát hành theo % (staged rollout)

Với Angular dev: giống deploy web nhưng có **người duyệt**, **chữ ký số**, và **không rollback tức thì** — bản lỗi đã cài trên máy người dùng. Vì vậy kiểm tra kỹ (Chương 5) và có cơ chế **force update**.

Ví dụ

Version — một nguồn duy nhất

`pubspec.yaml`: `version: 1.0.0+1`. Docs: iOS dùng phần trước + cho `CFBundleShortVersionString`, phần sau cho `CFBundleVersion`; Android dùng `versionName` / `versionCode`. Ghi đè khi build: `--build-name=1.0.1 --build-number=7`.

Build bản phát hành

```
# iOS (trên Mac): tạo .xcarchive + .ipa trong build/ios/ipa
flutter build ipa --release --obfuscate --split-debug-info=build/symbols
# Upload: mở build/ios/archive/Runner.xcarchive bằng Xcode → Distribute App,
# hoặc ứng dụng Transporter, hoặc:
xcrun altool --upload-app --type ios -f build/ios/ipa/*.ipa --apiKey <key_id> --
apiIssuer <issuer_id>

# Android: tạo .aab trong build/app/outputs/bundle/release/
flutter build appbundle --release --obfuscate --split-debug-info=build/symbols
```

NOT RUN trong sandbox (không có Xcode / Android SDK). Lệnh và cờ lấy từ docs iOS/Android và `flutter build appbundle -h` (sandbox có lệnh này và in ra mô tả `--obfuscate`, `--split-debug-info`).

Ký app Android (theo docs)

1. Tạo upload keystore: `keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -storetype JKS -keysize 2048 -validity 10000 -alias upload`.
2. Tạo `android/key.properties` (mật khẩu, alias, đường dẫn file) — **không commit** (`.gitignore` của sách đã chặn).
3. Sửa `android/app/build.gradle.kts` đọc `key.properties` và dùng `signingConfigs.release` thay cho `debug`. Docs còn có mục R8 (thu nhỏ code, bật sẵn cho bản release), và mục mới về ký lai hậu lượng tử (Android 17+) — xem docs khi cần.

Force update — examples/lib/chapters/ch06/force_update.dart

```
int compareVersions(String a, String b) {
  List<int> parse(String v) => v.split('+').first.split('.').map(int.parse).toList();
  final pa = parse(a), pb = parse(b);
  for (var i = 0; i < 3; i++) {
    final c = pa[i].compareTo(pb[i]);
    if (c != 0) return c;
  }
  return 0;
}

/// Bắt cập nhật khi bản đang cài thấp hơn bản tối thiểu (server/Remote Config trả
về).
bool needsForceUpdate({required String installed, required String minimumSupported})
=>
  compareVersions(installed, minimumSupported) < 0;
```

So sánh **từng phần số**: "1.10.0" > "1.9.9" (so chuỗi sẽ sai). Test ở phần Ch.5–6 của `test/chapters_test.dart` (pass).

Đi sâu

Icon và launch screen

Docs: icon iOS nằm trong `ios/Runner/Assets.xcassets/AppIcon.appiconset`; Android trong `android/app/src/main/res/mipmap-*`. Launch screen iOS: `LaunchScreen.storyboard`. Package `flutter_launcher_icons` / `flutter_native_splash` (2.4.8) sinh các file này từ một ảnh — tùy chọn, sách không dùng.

App Review — những điều hay bị từ chối (tổng quát)

Thiếu chính sách quyền riêng tư (privacy policy) khi thu thập dữ liệu, xin quyền mà không giải thích (Info.plist cần câu mô tả cho camera/micro/vị trí...), app "chỉ là trang web", crash khi duyệt. Chi tiết chính thức ở App Review Guidelines của Apple (chưa mở được trong sandbox — **UNVERIFIED**). App TOEIC dùng thông báo cục bộ và TTS: không cần quyền đặc biệt ngoài thông báo.

Privacy manifest / nhãn dữ liệu

Apple yêu cầu mô tả dữ liệu app thu thập (App Privacy trên App Store Connect); Google Play có mục Data safety. App offline như "Sổ Từ Vựng" / TOEIC thường khai "không thu thập dữ liệu" — nếu thêm Sentry (Chương 7) thì phải khai báo. **UNVERIFIED** về chi tiết biểu mẫu hiện tại.

Symbols và crash

Giữ thư mục `build/symbols` của **mỗi** bản phát hành (đặt tên theo version). Khi có stack trace đã obfuscate: `flutter symbolize -i <file stack trace> -d <file .symbols tương ứng trong build/symbols>` (docs "Obfuscate Dart code").

Web

`flutter build web --release` → thư mục `build/web` tải lên bất kỳ hosting tĩnh (Firebase Hosting, GitHub Pages...); docs "Build and release a web app". Đây là cách Nobin mở app trên iPhone bằng Safari mà không cần App Store (Tập 1, Ch.1).

Lỗi và bẫy thường gặp

- **Bundle ID / Application ID đổi sau khi phát hành** → thành app khác. Chọn kỹ ngay từ đầu (`--org` khi `flutter create`).
- **Mất upload keystore** → Play App Signing có quy trình reset nhưng mất thời gian; sao lưu keystore an toàn.
- **Quên tăng build number** → store từ chối.

- **Phát hành bản debug** hoặc còn `debugShowCheckedModeBanner` → xấu, chậm.
- **Xin quyền không giải thích** (thiếu `NS...UsageDescription`) → app crash khi xin quyền trên iOS / bị từ chối.
- **Không lưu symbols** → không đọc được crash.

Tóm tắt

- iOS: Apple Developer Program → Bundle ID → App Store Connect → `flutter build ipa` → TestFlight → App Review.
- Android: keystore + `key.properties` → `flutter build appbundle` → Internal testing → Production.
- Version từ `pubspec.yaml`; build number luôn tăng; obfuscate + giữ symbols; force update bằng so sánh version.

Bài tập (có lời giải)

Bài 1. Viết `compareVersions` sao cho `"1.10.0" > "1.9.9"` và bỏ qua phần build (+7). Vì sao không so sánh chuỗi?

▼ Lời giải

Code ở trên. So sánh chuỗi theo từ điển: `"1.10.0" < "1.9.9"` vì ký tự `'1' < '9'` ở vị trí thứ 3 → sai. Tách thành số rồi so từng phần. Test: `compareVersions('1.10.0', '1.9.9') > 0`, `compareVersions('1.2.3+7', '1.2.3+1') == 0`, `needsForceUpdate(installed: '1.2.0', minimumSupported: '1.3.0') == true`.

Bài 2. Viết checklist phát hành bản 1.0.0 của app TOEIC lên TestFlight (không cần code).

▼ Lời giải

1. `flutter pub outdated` + `flutter analyze` + `flutter test` + `scripts/check-all.sh` xanh.
2. `pubspec.yaml`: `version: 1.0.0+1`; Bundle ID cố định (ví dụ `dev.nobin.toEIC`).
3. Icon, tên hiển thị (`CFBundleDisplayName`), launch screen.
4. Kiểm tra trên iPhone thật: đọc to, nhắc hằng ngày, dữ liệu offline, dark mode (danh sách NOT RUN của sách).
5. Apple Developer Program đã kích hoạt; tạo app trên App Store Connect.
6. `flutter build ipa --obfuscate --split-debug-info=build/symbols/1.0.0+1`; lưu symbols.
7. Upload (Xcode Organizer / Transporter / `xcrun altool`); điền thông tin TestFlight, mời Nobin làm tester.
8. Ghi App Privacy (không thu thập dữ liệu nếu không có analytics/Sentry).

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Build and release an iOS app — <https://docs.flutter.dev/deployment/ios> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/ios.md>
- Build and release an Android app — <https://docs.flutter.dev/deployment/android> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/android.md>
- Build and release a web app — <https://docs.flutter.dev/deployment/web> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/web.md>
- Obfuscate Dart code — <https://docs.flutter.dev/deployment/obfuscate> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/deployment/obfuscate.md>
- Apple Developer Program: <https://developer.apple.com/programs/>
- Package flutter_native_splash: https://pub.dev/packages/flutter_native_splash
- Sách React Native trong repo: Tập 3, Chương 6 — Phát hành

Chương 7 — Monitoring: bắt lỗi toàn app, logging, báo cáo crash

Mục tiêu

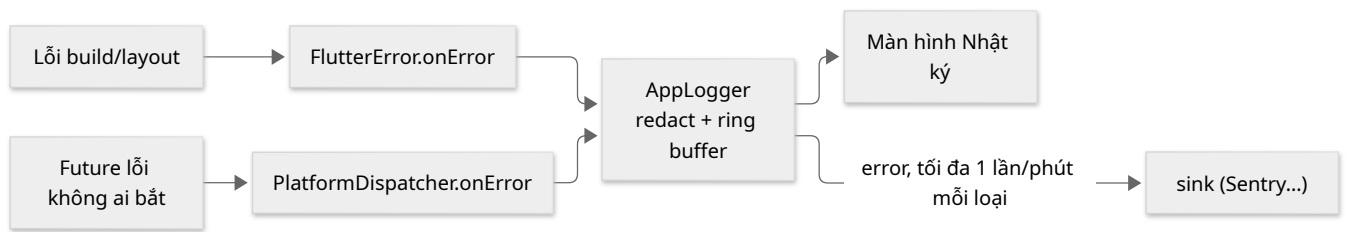
- Bắt **mọi** loại lỗi theo docs "Handling errors in Flutter": `FlutterError.onError` (lỗi framework), `PlatformDispatcher.instance.onError` (lỗi bất đồng bộ không ai bắt), `ErrorWidget.builder` (giao diện khi build lỗi).
- Viết **logger** có bộ đệm vòng (ring buffer), che dữ liệu nhạy cảm, chống "bão lỗi" khi gửi ra ngoài.
- Biết cách nối với dịch vụ crash reporting (ví dụ **Sentry**) — ở mức giới thiệu.
- Test được bộ bắt lỗi.

Giải thích đơn giản

Docs chia lỗi thành hai nhóm:

Loại lỗi	Ví dụ	Bắt bằng
Flutter bắt được — trong build, layout, paint, callback của framework	<code>null</code> trong build, overflow	<code>FlutterError.onError</code>
Flutter không bắt — lỗi bất đồng bộ	<code>onPressed: () async { await channel.invokeMethod(...) } ném lỗi, Future lỗi không ai await</code>	<code>PlatformDispatcher.instance.onError</code>

Với Angular: `FlutterError.onError` + `PlatformDispatcher.onError` ≈ một `ErrorHandler` toàn cục; `ErrorWidget.builder` ≈ trang lỗi dự phòng.



Ví dụ

Gắn bộ bắt lỗi — `examples/lib/core/logging/error_handlers.dart`

```
void installErrorHandlers(AppLogger logger) {
  final previous = FlutterError.onError;
  FlutterError.onError = (details) {
    logger.error('FlutterError: ${details.exceptionAsString()}', details.exception,
    details.stack);
    if (kDebugMode) previous?.call(details); // debug: vẫn in ra console như mặc định
  };
  PlatformDispatcher.instance.onError = (error, stack) {
    logger.error('Lỗi bất đồng bộ không được bắt', error, stack);
    return true; // đã xử lý → không làm app dừng
  };
  if (kReleaseMode) {
    ErrorWidget.builder = (details) => const Material(
      child: Center(child: Text('Đã có lỗi ở phần này. Hãy thử lại.')),
    );
  }
}
```

`main.dart` gọi `installErrorHandlers(logger)` **trước** `runApp`. Màn hình Cài đặt có nút "Gây lỗi thử" (`onPressed: () => Future<void>.error(StateError('Lỗi thử nghiệm'))`) để thấy lỗi bất đồng bộ hiện trong Nhật ký.

Logger — `examples/lib/core/logging/app_logger.dart`

```
void _add(LogRecord raw) {
  // Luôn che email / dãy số dài trước khi lưu hoặc gửi đi.
  final record = LogRecord(raw.time, raw.level, redact(raw.message), error: raw.error,
  stackTrace: raw.stackTrace);
  _records.add(record);
  if (_records.length > capacity) _records.removeAt(0);
  if (kDebugMode && record.level.index >= LogLevel.warning.index)
    debugPrint(record.toString());
  if (record.level == LogLevel.error && sink != null && _shouldSend(record)) sink!
  (record);
  notifyListeners();
}

/// Chống "bão lỗi": cùng một lỗi chỉ gửi ra ngoài tối đa 1 lần mỗi phút.
bool _shouldSend(LogRecord record) {
  final key = '${record.message}|${record.error.runtimeType}|${record.error}';
  final last = _lastSent[key];
  if (last != null && record.time.difference(last) < const Duration(minutes: 1))
    return false;
  _lastSent[key] = record.time;
  return true;
}
```

- `capacity` (mặc định 200): chỉ giữ N bản ghi gần nhất trong RAM.

- `sink`: nơi gửi lỗi ra ngoài (Sentry...); `null` = chỉ giữ trong máy (app của sách không gửi dữ liệu đi đâu).
- `AppLogger` extends `ChangeNotifier` → màn hình Nhật ký tự cập nhật (`context.watch<AppLogger>()`).

Test bộ bắt lỗi

```
test('installErrorHandlers: FlutterError và lỗi bất đồng bộ đều vào logger', () {
  final oldFlutter = FlutterError.onError;
  final oldPlatform = PlatformDispatcher.instance.onError;
  addTearDown(() {
    FlutterError.onError = oldFlutter;
    PlatformDispatcher.instance.onError = oldPlatform;
  });
  final logger = AppLogger();
  installErrorHandlers(logger);
  // ...
  FlutterError.reportError(FlutterErrorDetails(exception: Exception('lỗi build')));
  final handled = PlatformDispatcher.instance.onError!(StateError('lỗi async'),
    StackTrace.empty);
  expect(handled, isTrue);
  expect(logger.records.map((r) => r.level), [LogLevel.error, LogLevel.error]);
});
```

Luôn khôi phục handler cũ trong `addTearDown` — `flutter_test` dùng `FlutterError.onError` để phát hiện lỗi trong test; không khôi phục thì các test sau hoạt động sai. (Trong test này, sau khi gọi `installErrorHandlers`, sách gán lại `FlutterError.onError` bằng phiên bản không gọi `previous`, để lỗi giả không làm test thất bại.)

Kết quả thật (2026-09-30, `flutter test --reporter expanded test/logging_test.dart`):

```
00:00 +0: redact che email và dãy số dài
00:00 +1: AppLogger: giữ tối đa capacity bản ghi, che dữ liệu, chống gửi trùng trong 1 phút
00:00 +2: installErrorHandlers: FlutterError và lỗi bất đồng bộ đều vào logger
00:00 +3: Bài 1 (Ch.7): exportLogs lọc theo mức và che dữ liệu
00:00 +4: All tests passed!
```

Đi sâu

Nối với Sentry (giới thiệu, chưa dùng trong app)

`sentry_flutter` (9.30.1 trên pub.dev, 2026-09-30) tự gán các handler trên và gửi crash kèm stack trace lên dịch vụ Sentry (cần tài khoản + DSN). Với logger của sách, chỉ cần truyền `sink`:

```
final logger = AppLogger(sink: (r) => Sentry.captureException(r.error, stackTrace:
  r.stackTrace));
```

Đoạn này **chưa chạy** (không thêm package, không có DSN — NOT RUN). Khi dùng: khai báo trong App Privacy / Data safety (Chương 6), upload file symbols (bản obfuscate) để đọc được stack trace.

Log cái gì?

- **Có:** sự kiện quan trọng (mở app, tạo PIN, lỗi lưu dữ liệu), lỗi kèm ngữ cảnh (màn hình, thao tác).
- **Không:** PIN, token, nội dung ghi chú, email, số điện thoại. `redact()` là lưới an toàn cuối, không thay cho việc cẩn thận.
- **Mức:** `debug` (chỉ khi phát triển) < `info` < `warning` < `error`.

Zone và `runZonedGuarded`

Trước đây hay bọc `runApp` trong `runZonedGuarded` để bắt lỗi bất đồng bộ. Docs hiện hướng dẫn dùng `PlatformDispatcher.instance.onError`. Sách theo docs.

Theo dõi hiệu năng và sử dụng

DevTools (Chương 2) cho lúc phát triển. Ngoài thực tế: Firebase Crashlytics / Performance, Sentry Performance — cần cân nhắc quyền riêng tư. App học tập cá nhân (TOEIC) có thể chỉ cần logger nội bộ + nút "Gửi nhật ký" (Bài 1).

Lỗi và bẫy thường gặp

- **Gắn handler sau `runApp`** → bỏ lỡ lỗi khởi động.
- `PlatformDispatcher.onError` **trả false** → lỗi vẫn được coi là chưa xử lý.
- **Không khôi phục `FlutterError.onError` trong test** → test sau hỏng khó hiểu.
- **Log dữ liệu nhạy cảm** / gửi quá nhiều lỗi giống nhau → tốn quota và lộ dữ liệu.
- **Ẩn lỗi hoàn toàn ở debug** → khó sửa; sách vẫn in ra console khi `kDebugMode`.

Tóm tắt

- `FlutterError.onError` (lỗi framework) + `PlatformDispatcher.instance.onError` (lỗi async) + `ErrorWidget.builder` (UI dự phòng).
- **Logger:** ring buffer, `redact`, chống gửi trùng, `sink` cho dịch vụ ngoài.
- **Test** bộ bắt lỗi và nhớ khôi phục handler.

Bài tập (có lời giải)

Bài 1. Viết `exportLogs(records, minLevel)` xuất nhật ký thành văn bản ("HH:mm:ss [level] message — error") để người dùng gửi kèm báo lỗi; lỗi cũng phải được che dữ liệu.

▼ Lời giải

examples/lib/chapters/ch07/exercise_solution.dart:

```
String exportLogs(Iterable<LogRecord> records, {LogLevel minLevel =
LogLevel.info}) {
  String two(int n) => n.toString().padLeft(2, '0');
  return records
    .where((r) => r.level.index >= minLevel.index)
    .map((r) {
      final t =
        '${two(r.time.hour)}:${two(r.time.minute)}:${two(r.time.second)}';
      final err = r.error == null ? '' : ' - ${redact('${r.error}')}';
      return '$t [{r.level.name}] ${r.message}$err';
    })
    .join('\n');
}
```

Test: bản ghi debug bị lọc; lỗi `StateError('user a@b.co')` thành `Bad state: user <email>`.

Bài 2. Vì sao logger chống gửi trùng theo khóa `message|kiểu lỗi|lỗi` trong 1 phút thay vì gửi mọi lỗi?

▼ Lời giải

Một lỗi trong `build` có thể lặp lại **mỗi frame** (60 lần/giây) → gửi hàng nghìn bản giống nhau, tốn pin, mạng, quota dịch vụ. Gộp theo khóa giữ lại thông tin (lỗi xảy ra) mà không "spam". Test: gửi 2 lỗi giống nhau liên nhau → `sink` nhận 1; sau 2 phút → nhận thêm 1. Bản ghi vẫn được **lưu** đủ trong bộ đệm để xem trong máy.

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Handling errors in Flutter — <https://docs.flutter.dev/testing/errors> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/errors.md>
- Debugging Flutter apps — <https://docs.flutter.dev/testing/debugging> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/debugging.md>
- DevTools Logging view — <https://docs.flutter.dev/tools/devtools/logging> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/tools/devtools/logging.md>

- Package sentry_flutter: https://pub.dev/packages/sentry_flutter
- Sách React Native trong repo: [Tập 3, Chương 7 — Monitoring](#)

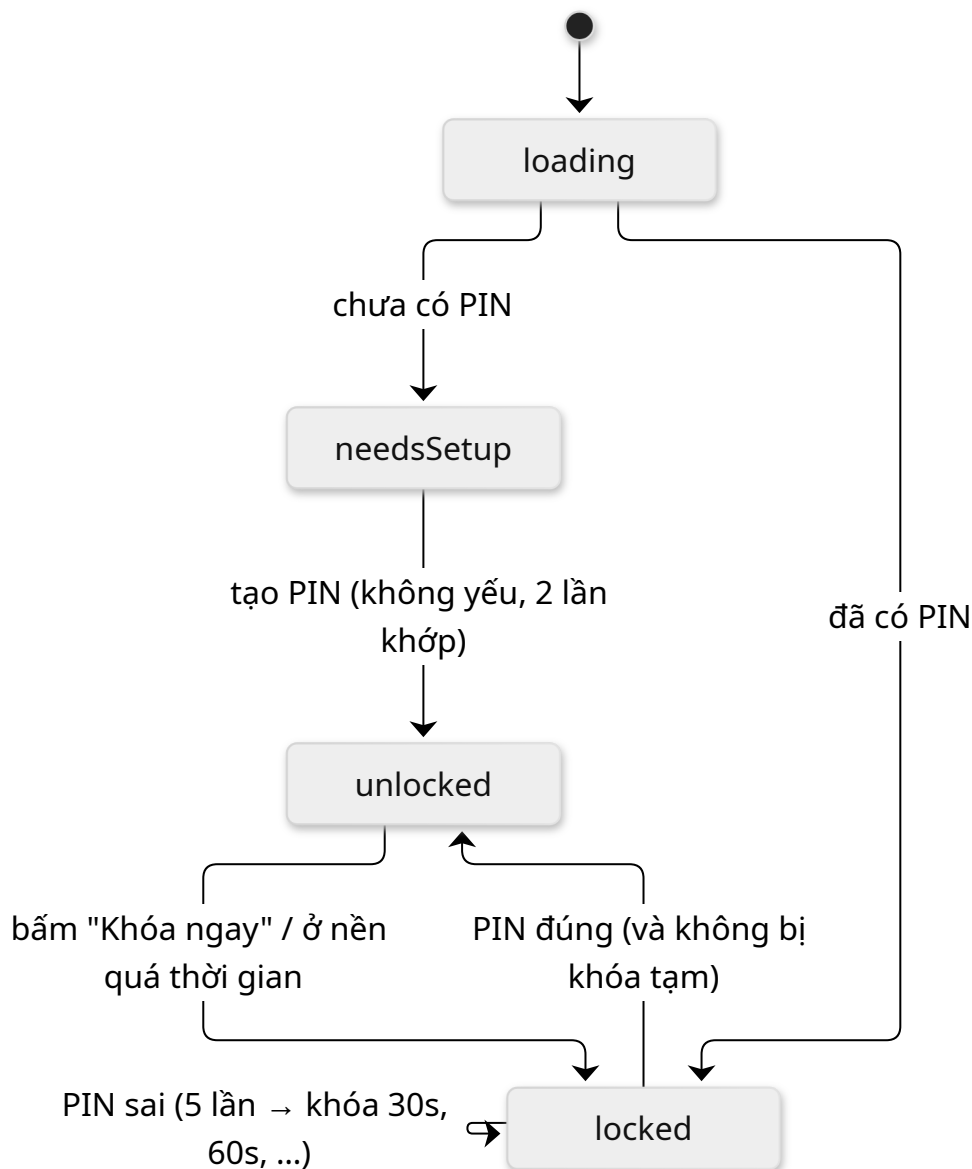
Chương 8 — App mẫu Tập 3: "Sổ Ghi Chú Bảo Mật"

Mục tiêu

- Ghép kiến thức Tập 3: kiến trúc **feature-first** có test luật phụ thuộc, **PIN + khóa tạm + tự khóa, secure storage, platform channel** (Swift/Kotlin), **bắt lỗi toàn app + logger**, performance, workflow CI mẫu.
- Chạy toàn bộ kiểm tra; biết phần nào đã kiểm chứng, phần nào NOT RUN.

Giải thích đơn giản

App ghi chú nhỏ, dữ liệu **mã hóa bởi hệ điều hành** (Keychain/Keystore), mở bằng mã PIN:



Router có `redirect`: trạng thái khác `unlocked` → mọi URL đều về `/lock`.

Ví dụ

Cấu trúc (xem Chương 1)

```
examples/lib/
  main.dart          installErrorHandlers(logger) →
runApp(SecureNotesApp(AppDependencies(...)))
  app.dart          MultiProvider + AppLifecycleListener (tự khóa) + MaterialApp.router
  router.dart       /lock, /notes, /notes/new, /notes/:id, /settings, /lab – guard bằng
redirect
  core/             result, command, logging (logger, error_handlers, redact),
platform/battery_channel,
                    security/secure_store, ui/theme
  features/auth/     domain/pin_hasher.dart · data/pin_repository.dart ·
ui/auth_controller.dart, lock_screen.dart
  features/notes/    domain/note.dart · data/note_repository.dart · ui/notes_viewmodel.dart,
notes_list_screen.dart, note_editor_screen.dart
  features/settings/ ui/settings_screen.dart (pin qua platform channel, thời gian tự khóa,
nhập ký)
  chapters/ch01..ch07/ ví dụ + lời giải
examples/ios/Runner/AppDelegate.swift          MethodChannel "dev.nobin.notes/battery"
examples/android/app/src/main/kotlin/.../MainActivity.kt MethodChannel cùng tên;
allowBackup="false"
vol3-nang-cao/ci/    flutter-ci.yml, ios-testflight.yml (mẫu)
```

Điểm vào

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  // sink: nơi gửi lỗi ra ngoài. Muốn dùng Sentry: sink: (r) =>
  Sentry.captureException(r.error, stackTrace: r.stackTrace)
  final logger = AppLogger();
  installErrorHandlers(logger);
  final store = FlutterSecureStore();
  runApp(
    SecureNotesApp(
      dependencies: AppDependencies(
        pins: PinRepository(store),
        notes: SecureNoteRepository(store),
        logger: logger,
        battery: const BatteryChannel(),
      ),
    ),
  );
}
```

Test toàn app — luồng chính

```
testWidgets('tạo PIN → ghi chú: thêm, sửa, xóa → khóa → mở khóa', (tester) async {
  final t = testDependencies();
  await tester.pumpWidget(SecureNotesApp(dependencies: t.deps));
  await tester.pumpAndSettle();
  expect(find.text('Tạo mã PIN'), findsOneWidget);
  // 1234 → "PIN quá dễ đoán..."; 2468 → vào danh sách
  // thêm "Mật khẩu wifi" → dữ liệu nằm trong SecureStore (key notes_v1)
  // sửa thành "Wifi nhà" → Khóa ngay → nhập sai 1357 → "Sai mã PIN (1 lần)" → nhập
  // đúng → thấy "Wifi nhà"
  // vuốt để xóa → "Chưa có ghi chú nào"
});
```

(Bản đầy đủ trong `examples/test/app_test.dart`; `testDependencies()` dùng `MemorySecureStore` và bấm PIN 10 vòng cho nhanh.)

Chạy

```
cd books/flutter/vol3-nang-cao/examples
flutter pub get
flutter run -d chrome      # web: secure storage dùng WebCrypto, chỉ chạy trên
localhost/HTTPS
flutter run                # iPhone (Mac + Xcode)
flutter analyze && flutter test
```

Kết quả kiểm tra thật (2026-09-30)

Test (`flutter test --reporter expanded`, gộp các file):

<code>auth_test.dart</code>	7 test	– bấm PIN, khóa tạm, tự khóa, AuthController
<code>notes_test.dart</code>	3 test	– SecureNoteRepository, NoteEditorViewModel, NotesViewModel
<code>platform_channel_test.dart</code>	4 test	– MethodChannel (mock native)
<code>logging_test.dart</code>	4 test	– redact, logger, bộ bắt lỗi, exportLogs
<code>chapters_test.dart</code>	9 test	– luật phụ thuộc, performance, deep link, PIN yếu, version
<code>app_test.dart</code>	4 test	– luồng chính, guard, cài đặt + nhật ký, Lab

`bash books/flutter/scripts/check-all.sh vol3-nang-cao (../logs/check-all.txt)`: YAML của `ci/` hợp lệ ("yaml OK: 2 file"), format OK, analyze "No issues found!", **31 test pass**, build web OK, smoke test Chromium thấy "Sổ Ghi Chú Bảo Mật" và "Tạo mã PIN" → "WEB SMOKE OK".

NOT RUN: chạy trên iPhone/Android; biên dịch `AppDelegate.swift` và `MainActivity.kt`; Keychain/Keystore thật; `AppLifecycleListener` trên thiết bị; workflow GitHub Actions; TestFlight.

Đi sâu

Vì sao ghi chú nằm trong secure storage thay vì SQLite?

Ít dữ liệu (vài chục ghi chú) → secure storage đủ và được mã hóa sẵn. Dữ liệu lớn hoặc cần tìm kiếm → SQLite mã hóa (SQLCipher) với khóa lưu trong Keychain — "Ideas for later". App TOEIC (Task 9) **không** cần mã hóa: từ vựng không nhạy cảm → SQLite thường (Tập 2).

Điều gì của Tập 3 nên mang sang app TOEIC?

- `core/logging` (`logger` + `installErrorHandlers`) — bắt lỗi toàn app ngay từ đầu.
- Test luật phụ thuộc (điều chỉnh cho cấu trúc kết hợp: `ui/` không import `data/services` trực tiếp...).
- Workflow CI mẫu (đổi đường dẫn) và `bumpBuildNumber` cho phát hành.
- Không cần PIN / secure storage trừ khi có đăng nhập.

Lỗi và bẫy thường gặp

- **Quên khôi phục handler lỗi trong test, kênh chưa mock trong widget test** (treo) — hai bẫy sách gặp thật khi viết app này.
- **Bấm PIN nhiều vòng trong test** làm test chậm → cho phép truyền `iterations` (app dùng 10.000, test dùng 10).
- **Router guard không có `refreshListenable`** → khóa rồi mà màn hình vẫn mở.
- **Web: secure storage trên HTTP thường** (không phải localhost/HTTPS) → lỗi (README của package).

Tóm tắt

- Feature-first + test luật phụ thuộc; PIN an toàn hợp lý (muối, bấm, constant-time, khóa tạm lưu bền, PIN yếu bị từ chối); tự khóa + guard; platform channel có test; logger + bắt lỗi toàn app.
- Kiểm chứng: analyze 0 issue, 31 test, build web, smoke web. Thiết bị thật: NOT RUN.

Bài tập (có lời giải)

Bài 1. Thêm luật vào test kiến trúc: `core/` không được import `features/` (đã làm ở Chương 1, Bài 1). Hãy giải thích vì sao luật này quan trọng với `AppLogger`.

▼ Lời giải

`AppLogger` được mọi feature dùng. Nếu nó import một feature (ví dụ để lấy `redact` từ `chapters/` hay `features/auth`), ta có vòng: `feature` → `core` → `feature`. Sách đã gặp đúng tình huống này và chuyển `redact` vào `core/logging/redact.dart`. Test "Bài 1: core không được import features" và "mã thật của app không vi phạm" (`chapters_test.dart`) giữ luật này mãi về sau.

Bài 2. Người dùng quên PIN. Đề xuất luồng "Quên PIN" an toàn cho app này (không cần code).

▼ Lời giải

Không có server nên **không thể** khôi phục ghi chú mà không có PIN (đó chính là mục đích mã hóa). Luồng an toàn: "Quên PIN" → cảnh báo rõ "tàn bộ ghi chú sẽ bị xóa" → xác nhận hai lần → xóa mọi khóa trong `SecureStore` (`pin_*`, `notes_v1`) → về màn hình "Tạo mã PIN". Nếu muốn khôi phục được, cần sao lưu mã hóa bằng một mật khẩu khôi phục riêng (hoặc Face ID qua `local_auth` làm phương án thứ hai) — "Ideas for later".

Nguồn tham khảo (Sources)

Đã mở ngày 2026-09-30 (đọc file nguồn docs ở commit `ab59c61`):

- Guide to app architecture — <https://docs.flutter.dev/app-architecture/guide> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/app-architecture/guide.md>
- Writing custom platform-specific code — <https://docs.flutter.dev/platform-integration/platform-channels> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/platform-integration/platform-channels.md>
- Handling errors in Flutter — <https://docs.flutter.dev/testing/errors> — <https://github.com/flutter/website/blob/ab59c614e780e2d6d44f07ae4a96238028f581a5/sites/docs/src/content/testing/errors.md>
- flutter_secure_storage 11.2.0: https://pub.dev/packages/flutter_secure_storage
- Sách React Native trong repo: Tập 3, Chương 8 — App mẫu Sổ Ghi Chú Bảo Mật

GLOSSARY — Thuật ngữ Flutter (Anh → Việt)

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
Flutter	Bộ công cụ UI của Google: một codebase Dart → app iOS, Android, web, desktop; tự vẽ mọi pixel	Angular (web) / React Native (mobile)
Dart	Ngôn ngữ lập trình của Flutter: có kiểu, null safety, biên dịch AOT/JIT/JS/Wasm	TypeScript
Widget	Mô tả bất biến (immutable) của một phần giao diện; "mọi thứ là widget"	Component
Widget tree (cây widget)	Các widget lồng nhau tạo thành cây	Cây component / DOM
StatelessWidget	Widget không có state riêng; chỉ phụ thuộc tham số	Component "dumb" / presentational
StatefulWidget	Widget có object State sống lâu, đổi bằng setState	Component có state
State (object)	Nơi giữ dữ liệu thay đổi và các hàm vòng đời initState, dispose	Class component + lifecycle hooks
build()	Hàm trả về cây widget ứng với state hiện tại	Template / render
setState()	Báo Flutter state đã đổi → build lại widget	signal.set() / setState của React
BuildContext	"Vị trí" của widget trong cây; dùng để tìm Theme, Navigator, provider ở trên	Injector hiện tại
Element	Đối tượng Flutter tạo từ widget để quản lý cây thật (ít khi dùng trực tiếp)	—
RenderObject	Đối tượng làm layout và vẽ (tầng thấp)	—
Key	Định danh giúp Flutter giữ đúng state khi danh sách đổi thứ tự	track trong @for / key của React
Constraints (ràng buộc)	Cha nói cho con kích thước tối thiểu/tối đa; con chọn kích thước trong đó	— (khác CSS)
Row / Column / Flex	Bố cục ngang / dọc	Flexbox flex-direction
Expanded / Flexible	Con chiếm phần còn lại của Row/Column	flex: 1
Material 3	Hệ thiết kế của Google, mặc định trong Flutter	Angular Material
Cupertino	Bộ widget kiểu iOS	—
ThemeData / ColorScheme	Cấu hình màu, chữ toàn app; hỗ trợ sáng/tối	Theme SCSS / CSS variables
Sliver	Mảnh của vùng cuộn (SliverList, SliverAppBar) cho hiệu ứng cuộn tùy biến	Virtual scroll (CDK)
Navigator	Ngăn xếp (stack) màn hình kiểu mệnh lệnh: push / pop	router.navigate + history
go_router	Package định tuyến theo URL (declarative), docs chính thức khuyên dùng	Angular Router
Deep link	Mở app thẳng vào một màn hình bằng URL	Route URL
Redirect (go_router)	Hàm chuyển hướng trước khi hiển thị route	Route guard canActivate

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
ShellRoute / StatefulShellRoute	Route "khung" bọc các route con (tab bar)	Layout component + <code><router-outlet></code>
Future	Một giá trị sẽ có trong tương lai	Promise
Stream	Chuỗi nhiều giá trị theo thời gian	Observable (RxJS)
<code>async</code> / <code>await</code>	Viết code bất đồng bộ như đồng bộ	Giống TypeScript
Isolate	"Luồng" riêng có bộ nhớ riêng để chạy việc nặng	Web Worker
ChangeNotifier	Class có sẵn, gọi <code>notifyListeners()</code> để báo đổi	Service có state + Subject/Signal
ValueNotifier	ChangeNotifier giữ một giá trị	<code>signal()</code>
ListenableBuilder / ValueListenableBuilder	Widget build lại khi Listenable báo đổi	<code>async pipe</code> / đọc signal trong template
InheritedWidget	Cơ chế truyền dữ liệu xuống cây, con tìm lên bằng context	Hierarchical injector
provider	Package DI + lắng nghe ChangeNotifier, docs chính thức khuyên dùng	Angular DI (<code>providers</code> , <code>inject</code>)
MVVM	Model–View–ViewModel: View = widget, ViewModel = ChangeNotifier	Component + service/facade
Repository	Lớp che giấu nguồn dữ liệu (SQLite, API) khỏi UI	Data service
Command (pattern)	Đối tượng bọc một hành động bất đồng bộ + trạng thái chạy/lỗi	NgRx effect đơn giản
Hot reload	Nạp lại code khi đang chạy, giữ state (phím <code>r</code>)	HMR / Fast Refresh
Hot restart	Khởi động lại app, mất state (phím <code>R</code>)	Tải lại trang
pub / pub.dev	Trình quản lý package và kho package của Dart	npm / npmjs.com
<code>pubspec.yaml</code> / <code>pubspec.lock</code>	Khai báo phụ thuộc / khóa phiên bản	<code>package.json</code> / <code>package-lock.json</code>
Plugin	Package có code native (Swift/Kotlin)	Native module (RN)
Platform channel (MethodChannel)	Kênh gọi qua lại giữa Dart và code native	TurboModule / bridge (RN)
Pigeon	Công cụ sinh code type-safe cho platform channel	Codegen (RN)
Swift Package Manager (SwiftPM)	Trình quản lý phụ thuộc iOS, mặc định từ Flutter 3.44	CocoaPods
AOT / JIT	Biên dịch trước (bản release) / biên dịch lúc chạy (debug, hot reload)	<code>ng build</code> / <code>ng serve</code>
Debug / Profile / Release mode	3 chế độ build: gỡ lỗi / đo hiệu năng / phát hành	dev / prod build
DevTools	Bộ công cụ gỡ lỗi, đo hiệu năng của Flutter	Chrome DevTools, Angular DevTools
Impeller	Engine vẽ mới của Flutter (mặc định trên iOS)	—

Thuật ngữ (English)	Giải thích tiếng Việt	Gần giống trong Angular / React Native
CanvasKit / skwasm	Bộ vẽ của Flutter web (Skia biên dịch sang WebAssembly)	—
Semantics	Cây thông tin trợ năng (VoiceOver đọc); test cũng tìm theo semantics	ARIA
Widget test	Test một widget trong môi trường giả, không cần thiết bị	Component test (TestBed)
Integration test	Test cả app trên thiết bị/giả lập (package integration_test)	E2E (Playwright/Cypress)
Fake / Mock	Bản giả tự viết (fake) / đối tượng giả kiểm tra lời gọi (mock, mocktail)	<code>jasmine.createSpyObj</code>
sqflite	Plugin SQLite cho iOS/Android	expo-sqlite
shared_preferences	Lưu cặp key-value nhỏ	localStorage / AsyncStorage
TTS (text-to-speech)	Đọc chữ thành giọng nói (flutter_tts)	expo-speech
Local notification	Thông báo do app tự đặt lịch, không cần server	expo-notifications (local)
Keychain / Keystore	Kho bí mật của iOS / Android (flutter_secure_storage)	expo-secure-store
Obfuscation	Làm rối tên hàm/lớp trong bản build để khó dịch ngược	Minify
Flavor	Biến thể build (dev/staging/prod)	Angular environments
TestFlight	Kênh phát bản thử iOS của Apple	—